StudyFlow AI — Software Requirements Specification

SRS v1.0 — Draft for Engineering Review

StudyFlow AI Group 03

04 November 2026

TABLE OF CONTENTS

Revision history....3
Approval record....3
Document map....4
1. INTRODUCTION....4
1.1 Purpose....4
1.2 Scope....4
1.3 Project background and problem statement....4
1.4 Objectives....4
1.5 Intended audience....5
1.6 Definitions and abbreviations....5
1.7 Source documents and references....5
2. OVERALL SYSTEM DESCRIPTION....6
2.1 Product perspective....6
2.2 Target users and user characteristics....6
2.3 Product functions by release....6
2.4 Operating environment....6
2.5 Constraints....6
2.6 Assumptions and dependencies....7
2.7 Product boundaries....7
3. SYSTEM ARCHITECTURE AND COMPONENT RESPONSIBILITIES....7
3.1 High-level architecture....7
3.2 Component responsibilities....8
3.3 Trust boundaries and data ownership....8
3.4 Main data/control sequence....9
3.5 Future integration model....9
4. RELEASE SCOPE AND FEATURE BOUNDARIES....10
4.1 Initial Release / F1 MVP — in scope....10
4.2 Explicitly out of F1 MVP / deferred....10
4.3 Change control....10
5. FUNCTIONAL REQUIREMENTS....10
5.1 System-wide requirements....11
5.2 F1 — Task capture and confirmation....11
5.3 F1 — PDF task extraction....12
5.4 F1 — Availability management....13
5.5 F1 — Automatic planning and scheduling....14
5.6 F1 — Study-plan retrieval....15
5.7 F1 — Rescheduling....15
5.8 F1 — Orchestrator and conversation management....16
5.9 F2 — Learning Assessment & Adaptive Revision (Post-MVP)....17
5.10 F3 — Daily AI Learning (Post-MVP)....17
6. INTENT CATALOG AND PARAMETER SPECIFICATION....18
6.1 Canonical vocabulary proposal....18
6.2 Parameter definitions....19
6.3 Two-stage command validation....19
7. BUSINESS RULES AND VALIDATION....19
7.1 Structural validation....19
7.2 Semantic validation....20
7.3 Domain validation....20
7.4 Scheduler algorithm contract....20

7.5 Interval semantics....21
7.6 Validation statuses....21
8. WORKFLOW AND STATE MANAGEMENT....22
8.1 Orchestrator processing stages....22
8.2 Workflow state definitions....22
8.3 State transition table....23
8.4 PendingAction lifecycle....23
8.5 Multi-intent dependencies....23
9. TOOL CONTRACTS AND API REQUIREMENTS....24
9.1 Contract envelope — illustrative only....24
9.2 Detailed minimum tool semantics....25
9.3 Error response envelope....25
10. DATA REQUIREMENTS AND DATA MODEL....26
10.1 F1 entities....26
10.2 Post-MVP entities from proposal....27
10.3 Relationships and cardinality....27
10.4 Data integrity and lifecycle....27
11. TIME AND DATE HANDLING....27
11.1 Data representation....27
11.2 Natural-language date/time interpretation....28
11.3 Vietnamese examples....28
12. ERROR HANDLING AND RECOVERY....28
12.1 Retry policy....29
12.2 Transaction and rollback....29
13. SECURITY AND AUTHORIZATION REQUIREMENTS....29
13.1 Threats and mitigations....30
14. NON-FUNCTIONAL REQUIREMENTS....30
15. EXTERNAL AND USER INTERFACE REQUIREMENTS....31
15.1 Student-facing interface....31
15.2 Existing proposal UI illustrations....32
15.3 Software interfaces....33
15.4 Communication and protocol decisions....33
16. USE CASE SPECIFICATIONS....33
16.1 UC-F1-01 — Capture, review and confirm an academic task....33
16.2 UC-F1-02 — Import a task candidate from an authorized PDF....33
16.3 UC-F1-03 — Declare or change availability....34
16.4 UC-F1-04 — Generate, inspect and approve a weekly study plan....34
16.5 UC-F1-05 — Retrieve a study plan....35
16.6 UC-F1-06 — Propose rescheduling of a confirmed study block....35
16.7 UC-F1-07 — Resolve clarification and pending action across turns....36
16.8 UC-F2-01 — Complete a verified quiz and review revision priorities (Post-MVP)....36
16.9 UC-F3-01 — Complete a source-grounded guided learning session (Post-MVP)....36
17. ACCEPTANCE CRITERIA....37
18. TESTING AND QUALITY ASSURANCE REQUIREMENTS....38
18.1 Test levels....38
18.2 Test scenario matrix....38
18.3 AI/NLU metrics....39
18.4 Test data governance....40
19. REQUIREMENT TRACEABILITY MATRIX....40
19.1 Traceability maintenance rule....41
20. OPEN DECISIONS, RISKS AND REQUIRED TEAM ACTIONS....41
20.1 Risk register....43
21. IMPLEMENTATION ROADMAP AND RELEASE GATES....43
21.1 P0 — Freeze requirements and design contracts....43
21.2 P1 — Implement and verify F1 MVP....44
21.3 P2 and Post-MVP....44
21.4 Team roles recorded by the proposal....44
APPENDICES....44
Appendix A — Canonical terminology proposed for implementation....44
Appendix B — Illustrative contract examples....45
Appendix C — Status taxonomy....47
Appendix D — Scheduler invariants and verification formulas....48
Appendix E — Assumption and TBD register....48
Appendix F — Source register and limitations....49
Appendix G — Final engineering readiness checklist....49

DOCUMENT CONTROL

Field	Value
Document title	StudyFlow AI — Software Requirements Specification
Document ID	SFAI-SRS-001
Version	1.0-DRAFT
Date	04 November 2026 (reference timezone: Asia/Ho_Chi_Minh)
Project / course	StudyFlow AI / CSC10014 — Computational Thinking
Institution	University of Science, Vietnam National University Ho Chi Minh City
Prepared for	StudyFlow AI Group 03
Document owner	TBD — team decision required
Reviewers / approvers	TBD — no approval evidence supplied
Primary implementation scope	F1 — Intelligent Academic Planner (Initial Release / MVP)
Future scope	F2 — Learning Assessment & Adaptive Revision; F3 — Daily AI Learning (Post-MVP)

Revision history

Version	Date	Change	Status
0.1	04 Nov 2026	Initial consolidation from proposal, context and worklist.	Draft
1.0-DRAFT	04 Nov 2026	Expanded implementation-level requirements, contracts, workflows, test scenarios and traceability.	For review

Approval record

Role	Name	Decision	Date
Project sponsor / instructor	TBD	Not approved	TBD
Project manager / team representative	TBD	Pending team review	TBD
Technical reviewer	TBD	Pending technical review	TBD

> **Draft status.** This document is an implementation-oriented specification draft. It does not claim that any feature has been implemented, tested, benchmarked or approved. Requirements labelled **PROPOSED** or **TBD** must be ratified before they become the team baseline.

HOW TO READ THIS SRS

This SRS uses the updated Group 03 Midterm Engineering Report as the primary source for product goals, release boundaries, feature descriptions, user flows and conceptual design. The two handoff files provide additional F1 intent, parameter, tool-contract, orchestrator and unresolved-work details. The earlier Master Prompt is used as a completeness checklist, not as a source of product facts.

Label	Meaning	Engineering treatment
BASELINE	Explicitly stated in Team03_Proposal_Updated(1).pdf.	Current project proposal baseline; confirm in team review before implementation freeze.
DESIGN	Present in context(2).md or cong_viec_can_lam(2).md as prior conceptual design.	Preserve, but do not assume formally approved or implemented.
PROPOSED	Derived technical specification needed to make behavior testable or codeable.	Review and approve before freezing.
TBD	Source does not decide the point or contains an unresolved conflict.	Do not silently invent a product decision; resolve via the decision register.
DEFERRED	Explicitly outside F1 MVP or planned after MVP.	Do not implement in F1 MVP unless scope changes are approved.

Normative terms: **SHALL** indicates mandatory behavior within the stated release scope; **SHOULD** indicates a recommended behavior; **MAY** indicates an optional capability. Normative wording does not turn an unapproved proposal into an approved project decision.

Document map

Sections	Content
1–2	Introduction, product context, objectives, users, environment and boundaries
3–4	Architecture, trust boundaries, release scope and feature ownership
5	Functional requirements: 74 requirements across system, F1, F2 and F3
6–8	Intent/slot contracts, business rules, validation and workflow/state management
9–12	Tool/API contracts, data model, date/time rules and error recovery
13–15	Security, non-functional requirements and external/UI interfaces
16–18	Use cases, acceptance criteria and test scenarios
19–21	Traceability, risks/decisions, roadmap, team roles and appendices

1. INTRODUCTION

1.1 Purpose

This document specifies the software requirements for StudyFlow AI, an intelligent academic learning assistant designed for undergraduate students. It defines externally visible behavior, component responsibilities, data contracts, business rules, state transitions, interfaces, security controls, quality requirements and verification conditions needed to implement and evaluate the system. It is intended as the shared implementation contract between frontend, backend, AI/NLU, QA and project coordination.

1.2 Scope

The product-level scope includes three connected student-facing features: **F1 Intelligent Academic Planner**, **F2 Learning Assessment & Adaptive Revision**, and **F3 Daily AI Learning**. The Initial Release / MVP includes only F1. F2 and F3 are planned Post-MVP core features. StudyFlow AI is a student support workflow, not a replacement for a full learning management system (LMS).

The F1 MVP covers manual task entry, candidate extraction from authorized text-based PDF files, student review and confirmation of task data, availability declaration, weekly study-plan generation, plan viewing and student-approved rescheduling. It must present infeasible or unscheduled work honestly. Confirmed schedules must not change without student approval and a successful backend commit.

1.3 Project background and problem statement

The proposal describes undergraduate students balancing several courses, assignments, deadlines, independent study and learning materials. Their coordination problem includes consolidating deadlines, identifying feasible study time, responding to changes and reducing manual information transfer among tools.

The proposal cites a team preliminary survey (n=17): 76.5% reported forgetting a deadline or completing work closer to it than planned during the previous four weeks; 70.6% selected study-time/deadline management as a time-consuming or difficult situation; 41.2% selected identifying weak topics; 76.5% selected finding explanations suitable for course materials; and 88.2% reported manually transferring information between study tools in the previous week. The report explicitly states that the questionnaire and anonymized response data were not supplied, so these figures cannot be independently audited from the report alone. They are contextual observations, not verified population estimates.

1.4 Objectives

ID	Objective	Release	Verification concept
OBJ-01	Generate feasible study schedules from student-confirmed tasks, deadlines and availability; expose unscheduled work.	F1 MVP	Hard-constraint tests and student review.
OBJ-02	Reduce manual coordination by supporting manual and authorized text-PDF task capture.	F1 MVP	Task capture and extraction-review tests.
OBJ-03	Allow students to inspect and approve proposed schedule changes.	F1 MVP	Approval, rejection, stale-proposal and commit tests.

ID	Objective	Release	Verification concept
OBJ-04	Assess students using source-grounded quizzes with verified answer keys and deterministic grading.	F2 Post-MVP	Human-reviewed question/key and grading evaluation.
OBJ-05	Provide source-grounded explanations and progressive hints, with student-confirmed session outcomes.	F3 Post-MVP	Evidence-support and outcome-confirmation tests.
OBJ-06	Coordinate later feature handoffs without silently changing confirmed plans.	Post-MVP integration	Handoff and authorization/approval tests.

1.5 Intended audience

- Project manager and team members coordinating requirements and release scope.
- Frontend developers implementing the web UI and confirmation workflows.
- Backend developers implementing task, availability, scheduling, persistence and APIs.
- AI/ML engineers implementing PDF text extraction, NLU/LLM adapters and later source-grounded features.
- QA engineers deriving unit, contract, integration, end-to-end and AI evaluation tests.
- Academic evaluators reviewing problem fit, design consistency, feasibility and evidence.

1.6 Definitions and abbreviations

Term	Definition
SRS	Software Requirements Specification.
MVP / Initial Release	Minimum viable first release; F1 is the only included feature.
LLM / NLU	Large Language Model / Natural Language Understanding component used for candidate interpretation or suitable language tasks.
Orchestrator	Shared coordination layer for routing, dialogue context, clarification, tool dispatch and safe handoffs.
Intent	Detected user goal, e.g. CREATE_TASK or GENERATE_STUDY_PLAN.
Slot / parameter	Named value extracted from a message, e.g. deadline_date or duration_minutes.
AcademicTask	A unit of academic work with a deadline and estimated/remaining effort; not itself a calendar interval.
PlannedStudyBlock	A specific start/end interval allocated to an AcademicTask.
StudyPlan	A set of proposed or confirmed study blocks for a planning period.
Availability	A time interval in which the student declares they can study.
Busy interval / commitment	A time interval that must not be used for planning; a dedicated FixedCommitment entity is not yet approved.
TaskCandidate	Unconfirmed task data from manual input or PDF extraction.
PendingAction	Persisted conversational workflow context awaiting clarification or another user decision.
ValidatedCommand	Normalized, entity-resolved and authorized operation input prepared by trusted application logic.
Proposal	A non-final draft of a change; it is not a confirmed schedule.
Approval evidence	A trusted record binding an explicit student decision to a specific proposal and version.
Commit	Transactional persistence of an approved and revalidated change.
Idempotency	Property ensuring retries of the same logical write do not create duplicate effects.
Half-open interval	Interval [start, end) that includes start and excludes end; adjacent blocks may touch without overlapping.
Post-MVP	Planned later release, not part of F1 Initial Release.

1.7 Source documents and references

1. Team03_Proposal_Updated(1).pdf — Midterm Engineering Report, StudyFlow AI, Group 03, October 2026, 28 pages. Primary source for project context and proposal baseline.
2. context(2).md — StudyFlow AI context handoff, version 1.1, 04 November 2026. Primary source for prior F1 design context and unresolved design details.

3. cong_viec_can_lam(2).md — current task register, 04 November 2026. Primary source for priorities, dependencies and unresolved work.
4. StudyFlow_AI_SRS_Master_Prompt(1).pdf — completeness and SRS structure guidance; not an independent source of project facts.

Source precedence. Use the updated proposal for project scope and stated product behavior. Use context/worklist for lower-level technical design only where it does not contradict the proposal. Record unresolved details as TBD or PROPOSED.

2. OVERALL SYSTEM DESCRIPTION

2.1 Product perspective

StudyFlow AI is a web application organized around a student-facing interface, a shared orchestration layer, three feature modules, and shared academic records plus authorized course materials. The first release activates F1 only. The architecture should preserve clean boundaries so F2/F3 can be added later without allowing their recommendations or session records to bypass F1 validation and student approval.

2.2 Target users and user characteristics

User class	Needs / characteristics	Release interaction
University student (primary and only confirmed actor)	Undergraduate student coordinating multiple courses, assignments, deadlines, weekly availability and course materials. Uses Vietnamese natural-language input and/or forms. Retains decision authority.	F1 MVP: tasks, PDF import, availability, plans and approvals. Post-MVP: quiz and guided learning.
Project team / operator (internal stakeholder)	Maintains, tests and evaluates the product; no separate administrative product role has been specified.	Internal development/operations; runtime permissions TBD.
Instructor / evaluator (academic stakeholder)	Reviews project scope, design, implementation evidence and evaluation. Not a confirmed runtime actor.	No application use case is mandated by supplied sources.

2.3 Product functions by release

Feature	Primary inputs	Primary outputs	Release
F1 — Intelligent Academic Planner	Manual task fields; authorized text-PDFs; availability; existing commitments; user constraints.	Confirmed AcademicTasks; proposed/confirmed StudyPlans; PlannedStudyBlocks; visible unscheduled work; rescheduling proposals.	Initial Release / MVP
F2 — Learning Assessment & Adaptive Revision	Authorized course PDFs; selected topic; reviewed source-grounded questions; student responses.	QuizAttempt; deterministic AssessmentResult; TopicPerformance; RevisionRecommendation.	Post-MVP
F3 — Daily AI Learning	Learning goal/question; approved study context; authorized course passages.	Source-grounded explanations/hints; reviewed summary; confirmed SessionOutcome.	Post-MVP

2.4 Operating environment

The proposal specifies a single web application. Browser support matrix, hosting platform, deployment topology, mobile responsiveness target, backend language/framework, database, object storage, authentication provider and LLM provider are not selected in the supplied materials. These are decision items, not known facts. Initial content-input support is text-based PDF extraction; OCR-heavy processing is excluded. Unreadable PDFs must fall back to manual entry.

2.5 Constraints

- F1 is the sole Initial Release / MVP feature. F2 and F3 must not become hidden dependencies for F1.
- Student confirmation is required for extracted essential task data before the task becomes eligible for scheduling.
- Proposed schedules and reschedules must be reviewed; autonomous schedule changes are not allowed.
- Scheduling constraints are enforced by deterministic application logic, not unconstrained LLM output.

- AI output is candidate/untrusted data and cannot set authenticated identity, authorize writes or prove approval.
- Only authorized course materials may be processed. Unrestricted external integrations and full LMS replacement are outside MVP.
- Voice/video tutoring, OCR-heavy processing, essay grading and predictive mastery modelling are outside the stated MVP scope.

2.6 Assumptions and dependencies

ID	Assumption / dependency	Status and consequence
ASM-01	A student identity can be authenticated and associated with owned academic records.	Necessary proposed assumption; authentication method TBD.
ASM-02	Course and task records can be isolated by authenticated owner.	Required security design; persistence model TBD.
ASM-03	The system can extract text from supported PDFs without OCR.	Baseline behavior in proposal; parser/library and file limits TBD.
ASM-04	A trustworthy timestamp, locale and timezone are available to the Orchestrator.	Context design; source and fallback policy TBD.
ASM-05	Busy intervals/commitments can be loaded by the planner.	Proposal requires commitment checks; representation/source unresolved.
DEP-01	F1 contract and scope decisions (P0) precede production implementation.	Worklist dependency.
DEP-02	F2/F3 depend on validated F1 and separate source/assessment/session ownership.	Post-MVP dependency.
DEP-03	Model selection follows contract/test dataset definition and benchmark.	No model/version currently approved.

2.7 Product boundaries

StudyFlow AI does not replace an LMS, guarantee academic success or mastery, autonomously alter a confirmed timetable, grade essays or predict mastery in the stated MVP. It does not assume unrestricted access to external calendars or institutional systems. Any such integration requires an explicit scope and authorization decision.

3. SYSTEM ARCHITECTURE AND COMPONENT RESPONSIBILITIES

3.1 High-level architecture

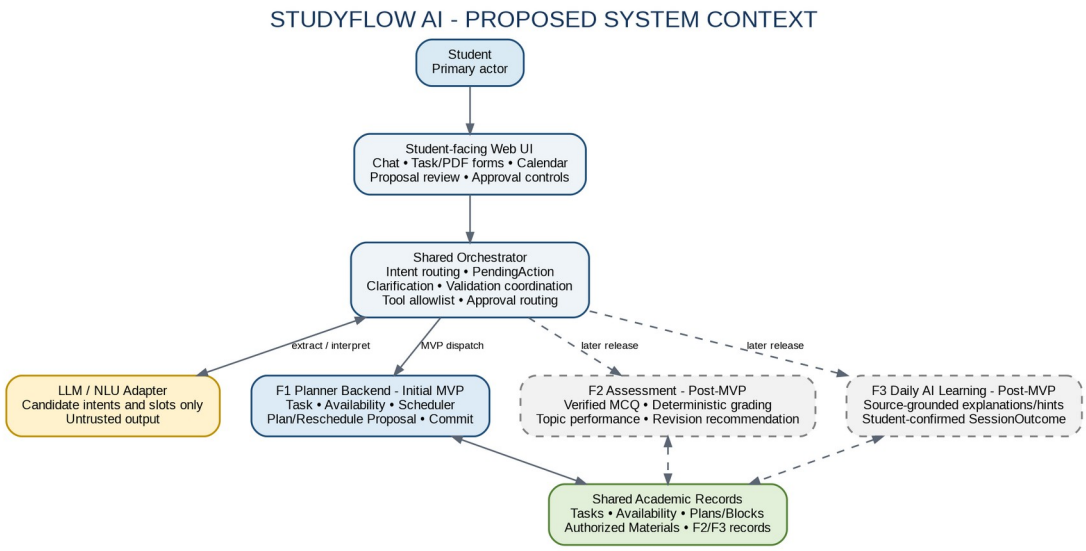


Figure 3-1. Proposed system context; dashed boxes/arrows indicate Post-MVP modules.

The logical architecture is: **Student-facing Web UI → Shared Orchestrator → F1 Planner Backend → Shared Academic Records**. LLM/NLU is an adapter invoked by the Orchestrator for candidate interpretation. F2/F3 are shown only to preserve the future system boundary.

3.2 Component responsibilities

Component	Responsibilities	Must not do
Student-facing Web UI	Capture messages/forms/files; show candidate task fields and provenance; display tasks, availability and weekly calendar; show proposed plan, unscheduled work, conflicts and confirmation controls; display operation status.	Treat a draft as committed; trust client-supplied owner identity or approval state.
LLM/NLU adapter	Classify intents; extract candidate slots; preserve original phrases; identify ambiguity and unsupported requests; optionally propose clarifications.	Select arbitrary tools/endpoints, invent IDs, assign permissions, validate ownership, schedule around hard constraints or commit data.
Shared Orchestrator	Load trusted request context; manage PendingAction; merge turns; coordinate validation; resolve references via backend; dispatch through allowlist; route approval to proposal; interpret ToolResult; recover safely.	Let raw model JSON reach privileged writes.
F1 Planner Backend	Authenticate/authorize; enforce domain validation; load tasks/availability/busy intervals; execute deterministic scheduling; produce plan/reschedule proposal; revalidate; commit transactionally; persist version/audit data.	Rely on model assertions for ownership, approval or schedule feasibility.
Shared Academic Records	Persist owned task/planning data and later feature records; enforce referential integrity and ownership boundaries.	Permit cross-user reads/writes or accept unvalidated bulk state from LLM.
F2 module (Post-MVP)	Retrieve authorized materials; generate candidate MCQs; verify source/key; grade verified items deterministically; aggregate topic performance; produce evidence-based recommendations.	Edit F1 schedule directly or infer mastery from small/insufficient samples.
F3 module (Post-MVP)	Retrieve relevant passages; generate source-grounded explanations/hints; manage learning session; ask student to confirm session summary.	Claim source support without evidence or save unconfirmed summary as confirmed outcome.

3.3 Trust boundaries and data ownership

- External user text, uploaded file contents and LLM/NLU output are untrusted inputs.
- Authentication identity, user ownership, authorization scope, timestamps and persisted entity IDs are trusted only when provided by server-side services.
- The Orchestrator validates extraction shape and routes only an allowlisted operation. The F1 Backend repeats all security-critical and domain checks.
- A plan proposal is not an approved plan. User approval must be associated with one proposal ID and exact version.
- A backend commit response is the source of truth for persistence status; UI state must follow it.

3.4 Main data/control sequence

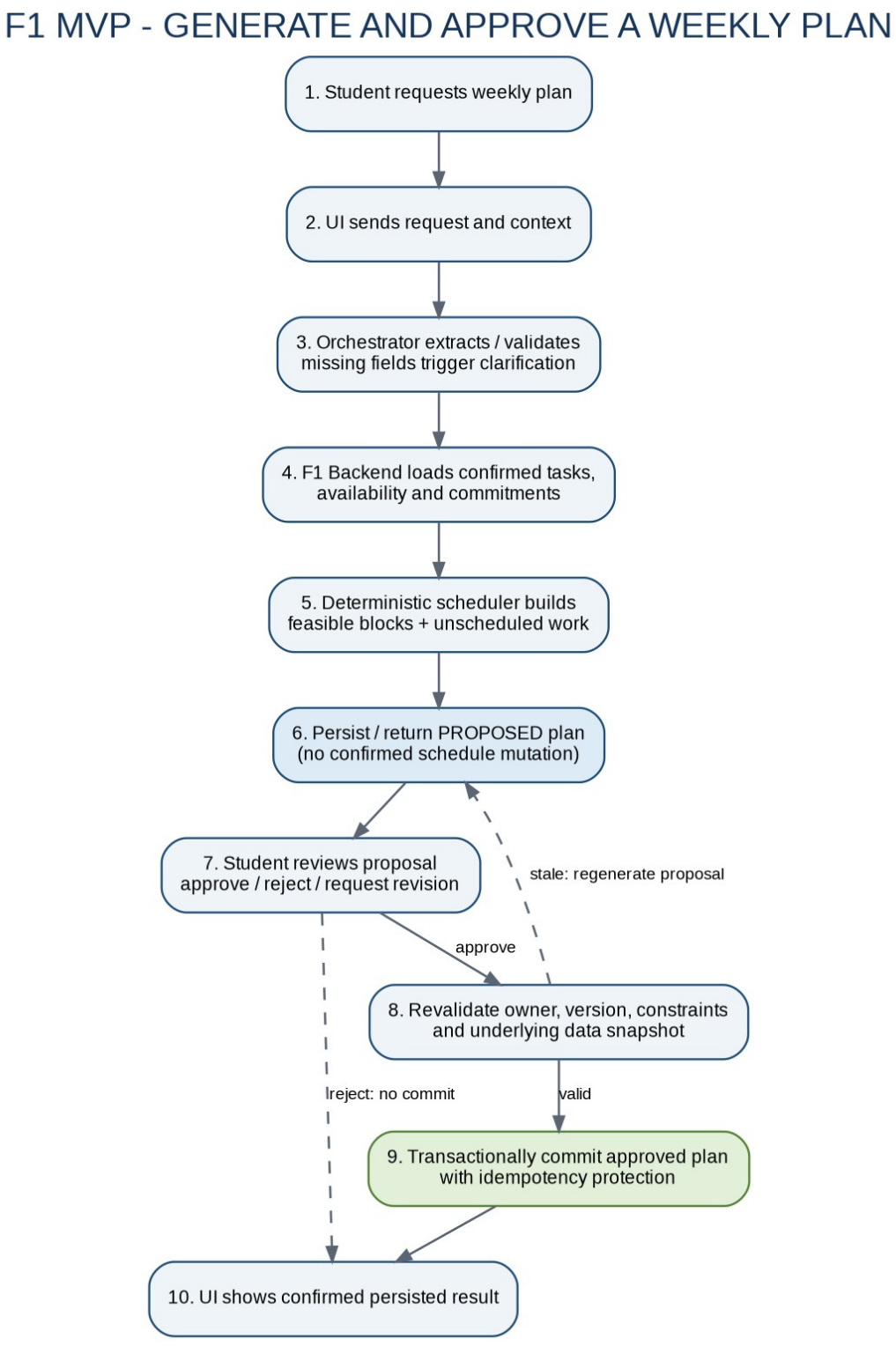


Figure 3-2. F1 weekly plan generation and student approval. Rejection has no schedule write; stale data requires a refreshed proposal.

3.5 Future integration model

After F1 validation, an approved PlannedStudyBlock may supply course/topic context to F3. F3 owns confirmed SessionOutcome records. A student may request an optional quiz from F3 or directly; F2 independently retrieves authorized sources and owns verified questions, QuizAttempt and AssessmentResult. F2 may return a RevisionRecommendation to F1. F1 asks the student to confirm workload and deadline, checks feasibility and generates another proposal. No handoff may mutate a confirmed schedule without

student approval. A SessionOutcome records activity and is not proof of mastery. A quiz is optional and does not have to follow a guided session.

4. RELEASE SCOPE AND FEATURE BOUNDARIES

4.1 Initial Release / F1 MVP — in scope

- Single web application for student-facing F1 workflows.
- Manual academic task entry and task candidate review/confirmation.
- Import authorized text-based PDF and extract candidate task fields; preserve source page/text provenance where available.
- Manual fallback when a PDF is unreadable or does not contain sufficient task details.
- Availability declaration, including date-specific or weekday/weekly patterns as supported by the approved recurrence policy.
- Weekly automatic plan generation from confirmed tasks and time constraints.
- Deterministic deadline-first scheduling; priority resolves equal-deadline ordering.
- Check availability, existing commitments, deadlines, cumulative workload and interval overlaps.
- Show a proposed plan and all unscheduled work; ask student to approve, reject or request revision.
- Retrieve and display existing plans and weekly calendar.
- Propose rescheduling of a planned block; preserve the current confirmed schedule until approval and successful commit.
- Vietnamese natural-language input and multi-turn clarification for missing/ambiguous information, as supported by the Orchestrator design.

4.2 Explicitly out of F1 MVP / deferred

Capability	Release treatment	Reason / notes
F2 quiz, grading and adaptive revision	DEFERRED — Post-MVP	Planned after F1 validation.
F3 guided learning, progressive hints and SessionOutcome	DEFERRED — Post-MVP	Planned after F1 validation.
UPDATE_TASK / DELETE_TASK	TBD scope; default defer from first implementation slice	Context/worklist identify as open; proposal does not detail these as core F1 MVP workflows.
CREATE_STUDY_BLOCK / manual placement	TBD scope; default defer	Context/worklist identify as scope-dependent; do not silently include.
Dedicated FixedCommitment entity	TBD data-model decision	Existing commitments must be checked, but a separate entity was not approved.
Advanced recurrence	DEFERRED / TBD	Only NONE/WEEKLY was discussed preliminarily; recurrence horizon and exceptions unresolved.
LMS replacement / broad calendar synchronization	OUT OF SCOPE	Explicit proposal boundary.
Autonomous schedule changes	OUT OF SCOPE	Student approval required.
Unrestricted external integrations	OUT OF SCOPE	Explicit proposal boundary.
Voice/video tutoring; OCR-heavy PDF processing	OUT OF SCOPE for MVP	Explicit proposal boundary; unreadable PDFs use manual fallback.
Essay grading; predictive mastery modelling	OUT OF SCOPE	Explicit proposal boundary.
Institution-wide administration or lecturer portal	TBD / not specified	No confirmed actor or use case in source.

4.3 Change control

Any feature moved into or out of MVP requires a recorded scope decision, impact assessment, updated requirement IDs/traceability and revised acceptance criteria. The existence of an intent name or mockup does not automatically mean the capability is part of MVP.

5. FUNCTIONAL REQUIREMENTS

The requirements below are atomic and testable. **BASELINE** means stated by the updated proposal; **DESIGN** means present in the F1 design handoff; **PROPOSED** means a concrete engineering rule added to make behavior implementable. Priority P0 is required to freeze the F1 design, P1 is implementation/release critical after P0 decisions, and P2 is deferred or Post-MVP.

5.1 System-wide requirements

ID	Requirement	Priority	Source / status	Verification
SYS-FR-001	The system SHALL provide a single student-facing web application for the in-scope Initial Release workflows.	P1	BASELINE	UI/E2E
SYS-FR-002	The system SHALL accept Vietnamese natural-language requests and structured form input for F1 workflows.	P1	DESIGN	E2E
SYS-FR-003	The system SHALL route every request through the Shared Orchestrator before invoking an F1 operation.	P0	DESIGN	Integration
SYS-FR-004	The system SHALL preserve the original user message and trusted request metadata (timestamp, locale and configured timezone) for request processing.	P0	DESIGN	Contract test
SYS-FR-005	The system SHALL distinguish READ, DRAFT, PROPOSE and COMMIT outcomes in its internal result and user-facing response.	P0	PROPOSED	Contract/E2E
SYS-FR-006	The system SHALL report implementation/release status only from actual persisted or test evidence and SHALL NOT represent mockups or proposals as implemented functionality.	P1	PROPOSED	Review/audit

5.2 F1 — Task capture and confirmation

ID	Requirement	Priority	Source / status	Verification
F1-TASK-001	The system SHALL allow a student to submit a task manually with title, course reference/name, start date, deadline, priority and estimated effort.	P1	BASELINE	UC-F1-01 / TC-01
F1-TASK-002	The system SHALL create task input as a candidate/draft until required fields have been validated and the student has confirmed the entered or extracted essential details.	P0	BASELINE	UC-F1-01 / TC-02
F1-TASK-003	The system SHALL retain candidate values separately from trusted, normalized task values until validation and confirmation complete.	P0	DESIGN	Contract test
F1-TASK-004	The system SHALL validate that task title is non-empty after trimming whitespace.	P1	PROPOSED	Unit test
F1-TASK-005	The system SHALL validate that estimated effort is a positive number of minutes before a task is eligible for scheduling.	P0	DESIGN	Unit test

ID	Requirement	Priority	Source / status	Verification
F1-TASK-006	The system SHALL validate that remaining effort, when supplied, is non-negative and does not exceed estimated effort.	P0	DESIGN	Unit test
F1-TASK-007	The system SHALL distinguish a task start date from its deadline date and SHALL NOT use the deadline field as a scheduled block end time.	P0	PROPOSED	Schema test
F1-TASK-008	The system SHALL resolve a task reference against records owned by the authenticated student before using it in an update, search-result action or rescheduling workflow.	P0	DESIGN	Security/integration
F1-TASK-009	The system SHALL ask the student to select a task when a name/reference resolves to multiple eligible records.	P1	DESIGN	TC-10
F1-TASK-010	The system SHALL make only confirmed, active tasks with positive remaining effort eligible for automatic scheduling.	P0	DESIGN	Scheduler unit test

5.3 F1 — PDF task extraction

ID	Requirement	Priority	Source / status	Verification
F1-PDF-001	The system SHALL accept an authorized PDF file as a task-capture input in the F1 MVP.	P1	BASELINE	UC-F1-02 / TC-03
F1-PDF-002	The system SHALL extract text from supported text-based PDF pages without requiring OCR.	P1	BASELINE	Parser integration
F1-PDF-003	The system SHALL produce task-field candidates from extracted text and SHALL identify which fields were not found or remain ambiguous.	P1	BASELINE	Extraction evaluation
F1-PDF-004	The system SHALL retain available source provenance for extracted candidates, including file reference and page number; text-span provenance SHOULD be retained where parser output permits.	P1	PROPOSED	Data/traceability test
F1-PDF-005	The system SHALL require student review and confirmation of essential extracted fields before the associated task becomes confirmed or schedulable.	P0	BASELINE	UC-F1-02 / TC-04
F1-PDF-006	When a PDF is unreadable, image-only, encrypted or yields insufficient text, the system SHALL explain the limitation and offer manual task entry rather than silently inventing values.	P1	BASELINE	TC-05

ID	Requirement	Priority	Source / status	Verification
F1-PDF-007	The system SHALL preserve the original file as an authorized source reference only according to an approved file-retention policy; until retention is decided, implementation SHALL use configurable retention and avoid indefinite duplication.	P1	PROPOSED / TBD	Security review

5.4 F1 — Availability management

ID	Requirement	Priority	Source / status	Verification
F1-AVL-001	The system SHALL allow a student to declare one or more availability intervals for a date or weekday.	P1	DESIGN	UC-F1-03
F1-AVL-002	The system SHALL capture a configured timezone for each availability interpretation and SHALL NOT rely on an LLM-invented timezone.	P0	DESIGN	Time contract
F1-AVL-003	The system SHALL validate that each availability interval has an end later than its start after timezone normalization.	P0	PROPOSED	Unit test
F1-AVL-004	The system SHALL support NONE and WEEKLY recurrence only if recurrence is approved for the MVP; recurrence end dates, exceptions and edits SHALL follow a policy recorded in the decision register.	P1	DESIGN / TBD	Recurrence tests
F1-AVL-005	The system SHALL detect when a new or changed availability interval conflicts with a confirmed study plan and SHALL NOT silently alter the confirmed plan.	P0	DESIGN	TC-18
F1-AVL-006	The system SHALL show the student the normalized availability interval before persisting any ambiguous or replacement availability value.	P1	PROPOSED	UI/E2E
F1-AVL-007	The system SHALL require explicit confirmation before overwriting existing availability when the change replaces or removes previously declared intervals.	P0	DESIGN	TC-19
F1-AVL-008	The system SHALL make availability intervals available to the deterministic scheduler only after successful persistence and authorization checks.	P0	PROPOSED	Integration

5.5 F1 — Automatic planning and scheduling

ID	Requirement	Priority	Source / status	Verification
F1-PLAN-001	The system SHALL allow a student to request a weekly study plan for an explicitly specified or policy-default planning period.	P1	BASELINE	UC-F1-04
F1-PLAN-002	The system SHALL load eligible confirmed tasks and the student's effective availability and existing busy intervals from trusted records before scheduling.	P0	BASELINE	Scheduler integration
F1-PLAN-003	The system SHALL sort eligible tasks by earlier deadline first and SHALL use task priority to resolve equal-deadline ordering.	P0	BASELINE	TC-14
F1-PLAN-004	The system SHALL use a deterministic scheduling procedure whose result is reproducible for the same task set, constraints, policy configuration and data snapshot.	P0	BASELINE	Determinism test
F1-PLAN-005	The system SHALL allocate study blocks only within the intersection of the planning period, task eligibility window and student availability.	P0	BASELINE / PROPOSED	Constraint test
F1-PLAN-006	The system SHALL exclude busy intervals and existing confirmed study blocks from available scheduling capacity.	P0	BASELINE	TC-15
F1-PLAN-007	The system SHALL enforce the half-open interval overlap rule: A and B overlap if start_A < end_B AND start_B < end_A.	P0	BASELINE	Boundary test
F1-PLAN-008	The system SHALL NOT allocate a block that extends beyond the applicable task deadline boundary.	P0	BASELINE	TC-16
F1-PLAN-009	The system SHALL account for cumulative workload and SHALL NOT allocate more minutes to a task than the amount remaining under the approved remaining-effort policy.	P0	BASELINE / TBD	Workload test
F1-PLAN-010	When available capacity is insufficient, the system SHALL return explicit unscheduled work with task reference, remaining minutes and a reason where determinable.	P0	BASELINE	TC-17
F1-PLAN-011	The system SHALL represent a generated plan as PROPOSED until valid student approval and successful commit have occurred.	P0	BASELINE	TC-20

ID	Requirement	Priority	Source / status	Verification
F1-PLAN-012	The system SHALL present proposed blocks, task associations, dates/times, total allocated effort and unscheduled work for student review.	P1	BASELINE	UI/E2E
F1-PLAN-013	The system SHALL preserve the current confirmed plan when a proposal is rejected, cancelled, expires or fails validation.	P0	BASELINE / PROPOSED	TC-21
F1-PLAN-014	The system SHALL record planning-period boundaries, policy/configuration version and relevant input-data version or fingerprint used to generate each proposal.	P0	PROPOSED	Persistence test

5.6 F1 — Study-plan retrieval

ID	Requirement	Priority	Source / status	Verification
F1-GET-001	The system SHALL allow the student to retrieve study plans for a requested date range.	P1	DESIGN	UC-F1-05
F1-GET-002	The system SHALL distinguish PROPOSED from CONFIRMED plans in returned data and UI presentation.	P0	BASELINE	TC-22
F1-GET-003	The system SHALL display the weekly calendar and associated blocks without treating a proposal as a confirmed calendar entry.	P1	BASELINE	UI inspection
F1-GET-004	When no plan exists for the requested period, the system SHALL return an empty result with a clear next action rather than an error.	P2	PROPOSED	Unit/E2E

5.7 F1 — Rescheduling

ID	Requirement	Priority	Source / status	Verification
F1-RSCH-001	The system SHALL allow the student to request a change to an identified confirmed PlannedStudyBlock.	P1	BASELINE	UC-F1-06
F1-RSCH-002	The system SHALL resolve the target block from authenticated student-owned records; duplicate or ambiguous references SHALL require selection.	P0	DESIGN	TC-10
F1-RSCH-003	The system SHALL retain the existing duration when the student changes only the target date/time and the block duration is known, unless the student explicitly requests a duration change.	P1	DESIGN	Unit/E2E

ID	Requirement	Priority	Source / status	Verification
F1-RSCH-004	The system SHALL validate a proposed block against availability, commitments, task deadline, task remaining effort and overlap constraints.	P0	BASELINE	TC-23
F1-RSCH-005	The system SHALL create a revised proposal and SHALL NOT modify the confirmed block until explicit approval and successful commit.	P0	BASELINE	TC-24
F1-RSCH-006	If constraints changed after proposal creation, the system SHALL reject stale approval and request a refreshed proposal.	P0	PROPOSED	TC-25

5.8 F1 — Orchestrator and conversation management

ID	Requirement	Priority	Source / status	Verification
F1-ORC-001	The system SHALL classify each F1 request into an allowlisted intent or mark it ambiguous/unsupported.	P0	DESIGN	NLU evaluation
F1-ORC-002	The system SHALL retain original user expressions for date, time, duration and task references alongside normalized candidate values.	P0	DESIGN	Contract test
F1-ORC-003	The system SHALL represent extraction output separately from a validated execution command.	P0	DESIGN	Contract test
F1-ORC-004	The system SHALL ask for missing or ambiguous information that blocks safe execution and SHALL NOT fabricate a deadline, effort, task identity or exact start time.	P0	BASELINE / DESIGN	TC-06 to TC-09
F1-ORC-005	The system SHALL merge short follow-up answers into the uniquely matching PendingAction; if multiple pending actions are plausible, it SHALL ask the student to identify the target.	P0	DESIGN	TC-26
F1-ORC-006	The system SHALL invalidate or supersede a proposal when a relevant field or underlying scheduling constraint changes.	P0	PROPOSED	TC-25
F1-ORC-007	The system SHALL dispatch only a ValidatedCommand through a deterministic intent-to-operation allowlist.	P0	DESIGN	Dispatch test
F1-ORC-008	The system SHALL route approval only when one active proposal ID and expected version are unambiguously identified.	P0	BASELINE / DESIGN	TC-27

ID	Requirement	Priority	Source / status	Verification
F1-ORC-009	The system SHALL interpret backend ToolResult status faithfully and SHALL NOT report a write as successful before the backend confirms persistence.	P0	PROPOSED	Failure injection

5.9 F2 — Learning Assessment & Adaptive Revision (Post-MVP)

ID	Requirement	Priority	Source / status	Verification
F2-FR-001	In a Post-MVP release, the system SHOULD retrieve topic-relevant passages from authorized text-based course materials with document/page/topic metadata.	P2	BASELINE — POST-MVP	Source traceability test
F2-FR-002	In a Post-MVP release, the system SHOULD draft candidate multiple-choice questions and answer keys grounded in retrieved passages.	P2	BASELINE — POST-MVP	Human review
F2-FR-003	Only reviewed and source-verified questions with verified answer keys SHALL be eligible for automatic grading.	P2	BASELINE — POST-MVP	Eligibility test
F2-FR-004	The system SHALL grade objective responses deterministically against verified answer keys and record a QuizAttempt and AssessmentResult.	P2	BASELINE — POST-MVP	Deterministic grading
F2-FR-005	The system SHALL calculate topic error rate only when at least one valid response exists and SHALL label a topic INSUFFICIENT_DATA when no valid responses exist.	P2	BASELINE — POST-MVP	Formula/boundary test

5.10 F3 — Daily AI Learning (Post-MVP)

ID	Requirement	Priority	Source / status	Verification
F3-FR-001	In a Post-MVP release, the system SHOULD retrieve relevant passages from authorized course materials for a student question or learning goal.	P2	BASELINE — POST-MVP	Retrieval evaluation
F3-FR-002	The system SHALL ground explanations and progressive hints in retrieved evidence and identify the limitation when the material does not support an answer.	P2	BASELINE — POST-MVP	Evidence support test
F3-FR-003	The system SHOULD preserve session context for follow-up questions until the student ends or the session expires under an approved policy.	P2	BASELINE — POST-MVP	Session test
F3-FR-004	The system SHALL present a session summary for student review before saving a confirmed SessionOutcome.	P2	BASELINE — POST-MVP	UC-F3-01

ID	Requirement	Priority	Source / status	Verification
F3-FR-005	The system SHALL NOT store a declined or unconfirmed summary as a confirmed SessionOutcome, and SHALL NOT treat a SessionOutcome as proof of mastery.	P2	BASELINE — POST-MVP	State/safety test

Functional requirement count: 74.

6. INTENT CATALOG AND PARAMETER SPECIFICATION

The catalog reflects the eight intent names discussed in the F1 context. An intent name is a semantic label, not proof that an API exists. CREATE_STUDY_BLOCK, UPDATE_TASK and DELETE_TASK remain scope-dependent and are not assumed to be F1 MVP operations.

Intent	Purpose	Required extraction fields (candidate)	Optional/context	Execution notes	Scope
CREATE_TASK	Create a task candidate from manual text or extracted PDF fields.	title; course reference/name; start_date; deadline_date; estimated_minutes	remaining_minutes; priority if defaults approved; source_ref	DRAFT candidate; validate and ask student to confirm. No scheduling while unconfirmed.	F1 MVP
UPDATE_TASK	Modify an existing confirmed task.	task_id; fields_to_update (at least one)	reason	Resolve owned entity and assess impact on linked blocks/plans before write.	TBD / default defer
DELETE_TASK	Remove or archive a task.	task_id	reason	Resolve owned entity; determine downstream plan/block behavior and confirmation policy.	TBD / default defer
SET_AVAILABILITY	Declare one or more available intervals.	date or weekday; start_time; end_time; timezone from trusted config	recurrence (NONE/WEEKLY preliminary)	Normalize interval; confirm replacements; do not silently alter confirmed plans.	F1 MVP
GENERATE_STUDY_PLAN	Generate a weekly plan proposal.	planning period start/end or approved default	selection_mode (ALL_ELIGIBLE / SELECTED_TASKS preliminary); task_ids when selected	Load eligible tasks and constraints from DB; return proposal plus unscheduled work.	F1 MVP
RESCHEDULE_STUDY_BLOCK	Propose a change to an existing confirmed block.	block_id; target date/time or constraints	duration; reason	Resolve block; preserve duration from record if not changed; propose only.	F1 MVP
GET_STUDY_PLAN	Retrieve plan(s) for a period.	date range or approved default	status_filter (CONFIRMED/PROPOSED/ALL preliminary)	Read-only; clearly separate proposal from confirmed plan.	F1 MVP
CREATE_STUDY_BLOCK	Manually place a study block for a task.	task_id; date; start_time; end_time or duration	plan_id	Must follow constraint validation and proposal/approval flow if approved for scope.	TBD / default defer

6.1 Canonical vocabulary proposal

Existing variants	Canonical proposal	Rule / migration note	Status
candidate_id / task_candidate_id	task_candidate_id	Use one stable field name in contracts; accept legacy alias only at adapter boundary if needed.	PROPOSED
parameters / slots	slots	Use slots for extracted conversational values; arguments for normalized operation input.	PROPOSED
end_date / deadline	deadline_date	Use deadline_date for the date-only task deadline; map legacy end_date at ingestion boundary.	PROPOSED

Existing variants	Canonical proposal	Rule / migration note	Status
duration / duration_minutes	duration_minutes	Integer minutes for a duration; do not confuse with a start/end timestamp interval.	PROPOSED
start_time/end_time	start_at/end_at	Use timezone-aware timestamps for persisted intervals; local input strings remain candidates until normalized.	PROPOSED
task_id / task_reference	task_id after resolution	LLM extraction may carry task_reference_text; only backend resolves to persisted task_id.	PROPOSED

6.2 Parameter definitions

Field	Type	Required when	Validation / notes
title	string	Task candidate creation	Trim whitespace; not empty; maximum length TBD.
course_id	opaque ID	When a course record is resolved	Must belong to authenticated student or be available in authorized scope.
course_name	string	When course_id is not yet resolved	Candidate label only; duplicate course names require selection.
start_date	date (YYYY-MM-DD)	Task scheduling	Inclusive eligibility start date; precise semantics proposed and need approval.
deadline_date	date (YYYY-MM-DD)	Task scheduling	Target completion date; exact day-end boundary policy must be approved.
estimated_minutes	integer	Task scheduling	Greater than zero.
remaining_minutes	integer	Task scheduling / replanning	0..estimated_minutes; update/progress source is TBD.
priority	enum or configured rank	Task scheduling	Exact enum and rank direction are TBD; stable tie-break required.
planning_period_start/end	date	Generate/retrieve plan	start <= end; MVP default length and week boundary TBD.
start_at/end_at	timezone-aware timestamp	Availability or block interval	end_at > start_at; normalized to a consistent timezone representation.
duration_minutes	integer	When user gives a duration	Greater than zero; end calculated only after start is unambiguous.
recurrence	enum/object	Availability recurrence	NONE/WEEKLY mentioned preliminarily; recurrence horizon/exceptions TBD.
task_reference_text	string	Natural-language extraction	Untrusted phrase; resolve by database query, never infer ID.
source_ref	object	PDF extraction	File reference, page number and extract span where available; authorized access required.

6.3 Two-stage command validation

Stage	Input	Permitted output	Owner / enforcement
Intent extraction	Original text + limited conversation context	Untrusted intent candidates, raw slot values, source phrases, ambiguity/missing fields, provenance	LLM/NLU adapter; no side effects
Validated command	Extraction result + authenticated context + DB lookups + validators	Allowlisted operation, normalized arguments, resolved entity refs, validation status	Orchestrator/shared validator; backend repeats domain/authorization checks
Execution result	Validated command	READ/DRAFT/PROPOSE/COMMIT/ ERROR result with IDs, versions and unscheduled work	F1 Backend; persisted state is authoritative

7. BUSINESS RULES AND VALIDATION

7.1 Structural validation

- JSON must parse and conform to the selected schema version.

- Enum values must be allowlisted; unknown fields are rejected or safely ignored according to a versioned compatibility policy (TBD).
- Date/time values must be normalized only after parsing; preserve original text for user-facing clarification.
- Numeric work estimates and durations must be integers in minutes and satisfy range constraints.

7.2 Semantic validation

Rule ID	Rule	Status
BR-STR-01	title is required for a task candidate and is not blank after trim.	PROPOSED
BR-SEM-01	estimated_minutes > 0.	DESIGN
BR-SEM-02	If remaining_minutes is provided: 0 <= remaining_minutes <= estimated_minutes.	DESIGN
BR-SEM-03	start_date <= deadline_date.	DESIGN; canonical name proposed
BR-SEM-04	An interval must have end strictly later than start after normalization.	PROPOSED
BR-SEM-05	A duration is positive; the system calculates an end only when start and timezone are unambiguous.	PROPOSED
BR-SEM-06	Unknown/missing task deadline or effort prevents scheduling unless an approved default/exception policy exists.	DESIGN / TBD
BR-SEM-07	A vague period such as morning/evening is not converted into an exact time without a user-approved policy or clarification.	DESIGN

7.3 Domain validation

Rule ID	Rule	Enforced by
BR-DOM-01	The entity must exist and belong to the authenticated user (or be explicitly shared under an approved policy).	F1 Backend
BR-DOM-02	Only confirmed eligible tasks are scheduled.	F1 Backend / Scheduler
BR-DOM-03	Candidate plan blocks must lie inside declared availability and outside busy intervals.	Scheduler
BR-DOM-04	No two blocks may overlap under half-open interval semantics.	Scheduler + DB constraint if selected
BR-DOM-05	A task block must not exceed its remaining schedulable workload.	Scheduler
BR-DOM-06	A task block must respect the task start/deadline window.	Scheduler
BR-DOM-07	When availability is insufficient, unscheduled work is returned; the algorithm must not fabricate capacity.	Scheduler
BR-DOM-08	An availability change conflicting with a confirmed schedule triggers explanation/replan; no silent mutation.	F1 Backend / Orchestrator
BR-DOM-09	Existing confirmed plans are not changed by proposal generation.	F1 Backend
BR-DOM-10	Before commit, the backend revalidates all constraints against current authoritative data.	F1 Backend

7.4 Scheduler algorithm contract

The proposal specifies a deterministic heuristic: earlier deadlines first; task priority breaks equal-deadline ties. It allocates within availability while checking existing commitments, deadlines, cumulative workload and overlap. The following pseudocode turns that behavior into a contract while leaving unresolved policy constants configurable.

INPUTS
planning_period
confirmed_tasks[]
availability_intervals[]
busy_intervals[]
confirmed_blocks[]
scheduler_policy (versioned configuration)

1. Validate the planning period and timezone.
2. Select confirmed tasks with positive schedulable remaining effort and eligibility intersecting the planning period.
3. Normalize all time intervals to one canonical instant representation.
4. Build free intervals = availability minus busy intervals minus existing confirmed blocks; use half-open intervals [start, end).
5. Sort tasks deterministically by:
   (deadline ascending, priority rank, stable task key)
   where priority rank direction and stable tie-break are policy decisions.
6. For each task in order:
   a. Intersect free intervals with the task eligibility window.
   b. Allocate chunks according to configurable block-splitting policy.
   c. Never exceed the task's schedulable remaining minutes.
   d. Update free intervals after every allocation.
7. Return proposed blocks and unscheduled minutes/reasons per task.
8. Persist/return as PROPOSED; do not alter the confirmed schedule.
9. On approval, re-run all validations against current data before commit.

The following policy values are not decided in the sources and must remain configuration parameters until approved: week start day; default planning period; deadline end-of-day cutoff; priority enum/rank; block minimum/maximum/preferred duration; block splitting; daily workload cap; breaks/buffers; distribution strategy across days; treatment of already scheduled but incomplete workload; and deterministic final tie-break fields.

7.5 Interval semantics

Canonical overlap test. For half-open intervals A=[sA,eA) and B=[sB,eB), $\text{overlap}(A,B) \Leftrightarrow sA < eB \text{ AND } sB < eA$. Adjacent intervals such as [09:00,10:00) and [10:00,11:00) do not overlap. This rule is stated in the updated proposal and references PostgreSQL OVERLAPS semantics.

7.6 Validation statuses

Status	Meaning	Next action
READY	Candidate data passes structural/semantic checks and enough context exists to prepare a command.	Continue to deterministic dispatch; READY is not approval.
NEEDS_CLARIFICATION	One or more fields or entity references are missing/ambiguous.	Create/update PendingAction and ask a focused question.
UNSUPPORTED	Request is outside the currently supported feature/scope.	Explain limitation and offer supported fallback.
INVALID	Supplied value or relationship violates schema/domain constraints.	Return validation details; ask for correction when possible.

Validation status is not the same as workflow state (e.g. AWAITING_APPROVAL) or entity status (e.g. PROPOSED/CONFIRMED).

8. WORKFLOW AND STATE MANAGEMENT

8.1 Orchestrator processing stages

Stage	Responsibility	Output / control
1. Input processing	Preserve original message; attach trusted timestamp, locale, timezone and authenticated user context; inspect pending actions.	UserMessage; context reference.
2. Intent classification	Identify single, multiple, ambiguous or unsupported intent.	Intent candidates, not commands.
3. Slot extraction	Extract candidate values and original phrases; attach provenance.	ExtractionResult.
4. Context management	Match follow-up to pending action; handle edits, cancellation, topic changes, expiry and version.	Updated PendingAction or new action.
5. Validation	Structural, semantic and domain validation; resolve entities through backend.	READY / NEEDS_CLARIFICATION / UNSUPPORTED / INVALID.
6. Clarification	Ask minimum question that unblocks execution; do not guess missing dates/effort/start times.	User response merged into pending action.
7. Tool dispatch	Select operation from static allowlist using ValidatedCommand.	Tool invocation with trusted context.
8. Response handling	Translate backend result; distinguish read, draft, proposal, commit and error.	User-facing response consistent with persisted status.
9. Confirmation & persistence	Collect explicit approval; bind it to proposal/version/user; revalidate then commit.	Confirmed persisted records or safe rejection.
10. Error recovery	Map error to clarification, limited retry, replan or manual fallback.	No duplicate/stale writes.

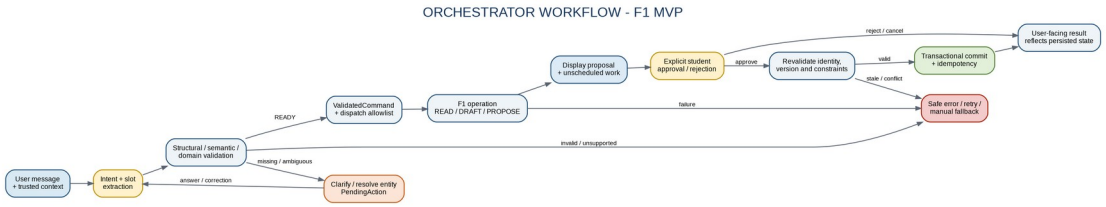


Figure 8-1. Orchestrator control flow with clarification loop and proposal/approval boundary.

8.2 Workflow state definitions

State	Description	Terminal?
RECEIVED	A new user message has entered the Orchestrator.	No
EXTRACTING	Intent and candidate slots are being produced.	No
VALIDATING	Structural, semantic and domain checks are running.	No
AWAITING_CLARIFICATION	Workflow waits for a required answer or entity selection.	No
READY_TO_DISPATCH	A validated, allowlisted command is ready.	No
EXECUTING	Backend operation is in progress.	No
AWAITING_APPROVAL	A proposal is available and awaiting a specific student decision.	No
COMMITTING	Backend is revalidating and transactionally applying approved changes.	No
COMPLETED	Operation completed; result reflects backend state.	Yes
REJECTED	Student rejected a proposal; no proposal changes committed.	Yes
CANCELLED	Pending workflow was cancelled.	Yes
EXPIRED	Pending action or proposal passed its expiry policy.	Yes
FAILED	Unrecoverable or non-retryable error occurred.	Yes
UNSUPPORTED	Request is outside current supported scope.	Yes

8.3 State transition table

Current state	Event	Guard	Action	Next state	Side effect
RECEIVED	Message accepted	Trusted user context exists	Preserve message; inspect PendingAction	EXTRACTING	No write
EXTRACTING	Extraction result	Schema invalid	Reject model output; bounded retry/fallback	FAILED or clarification	No write
EXTRACTING	Candidate intent/slots	Extraction available	Run validators	VALIDATING	No write
VALIDATING	Missing/ambiguous field	Required info unresolved	Create/update PendingAction; ask question	AWAITING_CLARIFICATION	No write
AWAITING_CLARIFICATION	User replies	One pending action matches uniquely	Merge answer with provenance; revalidate	EXTRACTING / VALIDATING	No write
AWAITING_CLARIFICATION	User replies	Multiple pending actions match	Ask user to choose target action	AWAITING_CLARIFICATION	No write
VALIDATING	Valid command	All checks pass	Create ValidatedCommand	READY_TO_DISPATCH	No write
READY_TO_DISPATCH	Dispatch	Operation is allowlisted	Call backend with trusted identity	EXECUTING	Depends on operation type
EXECUTING	READ result	Backend returns success	Format retrieved data	COMPLETED	Read only
EXECUTING	DRAFT result	Candidate persisted as draft	Display fields and request confirmation as required	AWAITING_APPROVAL or COMPLETED	Draft only
EXECUTING	PROPOSE result	Proposal created	Display proposal, version and unscheduled work	AWAITING_APPROVAL	No confirmed-plan mutation
AWAITING_APPROVAL	Approve	Exact proposal/version/user match; not expired	Create approval evidence and request backend revalidation	COMMITTING	Not yet committed
AWAITING_APPROVAL	Reject/cancel	Target proposal uniquely known	Record decision/cancel; preserve old confirmed state	REJECTED / CANCELLED	No commit
COMMITTING	Revalidation passes	Current data still compatible	Commit within transaction using idempotency	COMPLETED	Write allowed
COMMITTING	Stale/conflict	Version or constraints changed	Reject approval; regenerate proposal or clarify	FAILED / AWAITING_APPROVAL	No stale commit
Any nonterminal	Expiry	Expiry policy reached	Mark action/proposal expired	EXPIRED	No write
Any nonterminal	Topic change	New intent is not a continuation	Keep/cancel pending action per policy; do not merge unrelated message	RECEIVED	No unintended write

8.4 PendingAction lifecycle

- Create PendingAction only when a workflow must survive a user turn (clarification, entity selection or approval).
- Persist intent, collected slots, missing/ambiguous fields, original expressions, provenance, workflow state, version, owner, created_at and expires_at.
- A user correction increments the action version and triggers revalidation; relevant edits invalidate any derived proposal.
- A short answer such as “19h” may be merged only when one pending action clearly requests a start time and no competing action is plausible.
- “Đồng ý” may approve only one uniquely identified active proposal; otherwise ask which proposal the student means.
- Expiry, cancellation, rejection and topic-switch behavior must be explicit; never auto-commit on expiry.

8.5 Multi-intent dependencies

For MVP, process only explicit, safely orderable intents. Example: “Tôi rảnh thứ Hai 08:00–11:00, lập lịch tuần sau.” The system must first normalize and confirm the availability change if required by replacement/conflict policy, persist it, then generate the weekly plan using the persisted availability. The second operation must not consume an unconfirmed availability candidate. Complex parallel workflows are outside current scope.

9. TOOL CONTRACTS AND API REQUIREMENTS

The following are logical backend operations. The supplied documents do not select HTTP routes, HTTP methods, RPC transport, authentication protocol, framework or versioning scheme. Names below are operation identifiers, not claims of existing endpoints.

Operation	Type	Purpose	Input contract	Output contract
create_task_candidate	DRAFT	Create candidate task from manual/PDF extraction.	Authenticated user; candidate fields; provenance.	candidate_id, DRAFT status, missing fields, validation errors, confirmation_required.
confirm_task	COMMIT / state transition	Promote valid candidate to confirmed AcademicTask.	candidate_id, expected candidate version, explicit confirmation evidence.	task_id, CONFIRMED status, canonical fields.
search_tasks	READ	Search owned tasks by query/course/status.	query and filters, pagination.	Task summaries plus disambiguation candidates; no cross-user leakage.
set_availability	DRAFT/COMMIT (policy TBD)	Create or replace availability intervals.	Date/weekday, start/end, recurrence, trusted timezone, expected version if replacing.	Normalized availability and impact/conflict warning; persisted only under approved confirmation policy.
generate_study_plan	PROPOSE	Generate weekly schedule proposal.	Planning period, selection mode, optional owned task_ids.	plan_id, version, PROPOSED status, blocks, unscheduled work, input snapshot/version.
get_study_plan	READ	Retrieve proposal or confirmed plan.	Date range, status filter.	Plan metadata and blocks, explicitly separated by status.
propose_reschedule	PROPOSE	Create revised proposal for an existing block.	Owned block_id, target time/constraints, reason optional.	New proposal/version, proposed block changes, conflicts/unscheduled impact.
approve_study_plan	COMMIT	Approve and persist exact proposal version after revalidation.	plan_id, expected_version, approval evidence/reference, idempotency key.	CONFIRMED result, new version, committed blocks; or structured stale/conflict error.
reject_proposal	DECISION (proposed logical op)	Record rejection and keep confirmed state unchanged.	proposal_id, expected_version, explicit reject event.	REJECTED status; no plan mutation.
cancel_pending_action	CONTROL (proposed logical op)	Cancel a unique pending action.	pending_action_id, expected_version.	CANCELLED status; no domain write.
extract_task_candidates_from_pdf	DRAFT / AI adapter	Extract candidate fields from authorized text-PDF.	file_ref, user scope, extraction options.	Candidates with page/text provenance, missing fields and confidence/uncertainty metadata; exact confidence policy TBD.

9.1 Contract envelope — illustrative only

```
{
 "schema_version": "1.0",
 "operation": "generate_study_plan",
 "arguments": {
  "period_start": "2026-11-09",
  "period_end": "2026-11-15",
  "selection_mode": "ALL_ELIGIBLE",
  "task_ids": []
 },
 "context_ref": "trusted-request-context-reference"
}
```

```
{
 "status": "PROPOSED",
 "operation": "generate_study_plan",
 "data": {
  "plan_id": "opaque-server-generated-id",
  "version": 1,
  "study_blocks": [],
  "unscheduled_work": [],
  "requires_approval": true
 },
 "errors": []
}
```


Trust rule. The context reference, authenticated user ID, entity ownership and approval evidence are supplied/resolved by trusted services. They must not be accepted from arbitrary LLM-generated JSON or the browser as authoritative values.

9.2 Detailed minimum tool semantics

Operation	Validation / authorization	Side effects and idempotency
create_task_candidate	Authenticated owner; title/field schema; candidate source access.	Creates draft/candidate only. Retries with same request idempotency key should not create duplicate candidates; key strategy TBD.
confirm_task	Candidate owned by user; candidate version current; required fields valid; explicit confirmation.	Promotes or creates confirmed task; duplicate confirmation returns same confirmed task or safe conflict.
search_tasks	Authenticated owner filter mandatory; query never broadens data scope.	Read only; deterministic sort/pagination policy TBD.
set_availability	Validate timezone/interval/recurrence; check replacement semantics and conflict with confirmed plan.	Must not overwrite without explicit confirmation where replacement occurs; conflict triggers replan, never silent mutation.
generate_study_plan	Period valid; task IDs owned; load only eligible confirmed tasks; deterministic scheduler.	Creates PROPOSED plan; does not replace active confirmed plan. Repeated request semantics/versioning TBD.
get_study_plan	Authenticated owner and date range.	Read only.
propose_reschedule	Block owned and active; duration/task constraints resolved; domain validation.	Creates a new proposal referencing current plan version; no confirmed block mutation.
approve_study_plan	Exact proposal ID/version, user, expiry, one-time decision, fresh constraint validation.	Transactional commit; idempotency required; stale proposal rejected; duplicate approval returns prior result, not duplicate blocks.
reject_proposal	Exact proposal/version and owner.	Records rejection, no commit; duplicate rejection idempotent.
cancel_pending_action	Exact pending action/version and owner.	Marks workflow cancelled; no domain data write.

9.3 Error response envelope


```
{
 "status": "ERROR",
 "error": {
  "code": "STALE_PLAN_VERSION",
  "message": "The proposal is no longer current.",
  "details": {"plan_id": "...", "expected_version": 2, "current_version": 3},
  "retryable": false,
  "correlation_id": "..."
 }
}
```

10. DATA REQUIREMENTS AND DATA MODEL

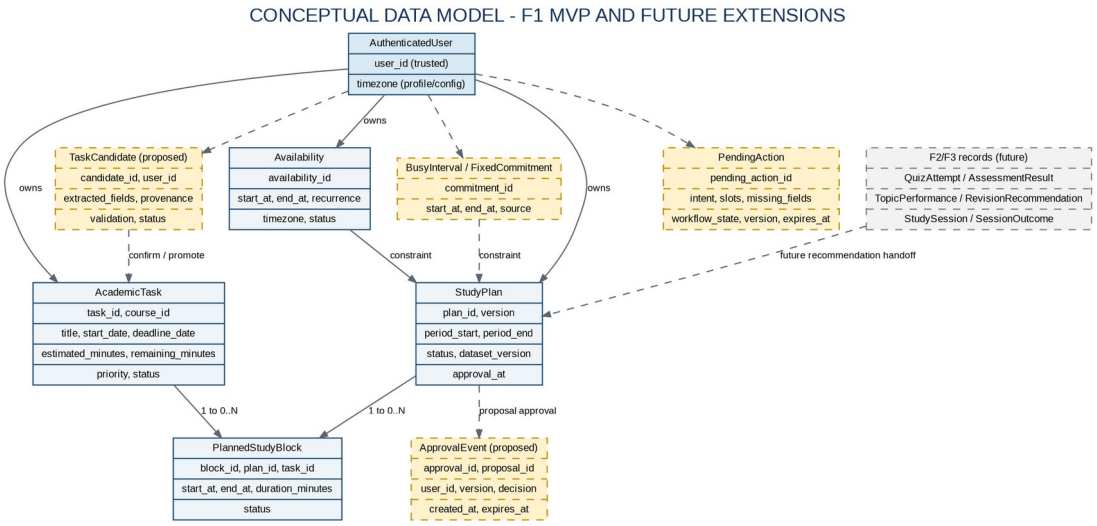


Figure 10-1. Conceptual entity model. Dashed entities/relationships are proposed or Post-MVP.

10.1 F1 entities

Entity	Scope/status	Minimum attributes	Semantics / constraints
AcademicTask	F1 core	task_id, owner_id, course_id, title, start_date, deadline_date, estimated_minutes, remaining_minutes, priority, status, created_at, updated_at	Task is academic work, not a calendar interval. Only confirmed eligible records enter scheduler. deadline_date is proposed canonical name for source end_date/deadline.
TaskCandidate	Proposed F1 helper	candidate_id, owner_id, source_type, candidate_fields, missing_fields, validation_status, provenance, version, created_at, expires_at	Temporary candidate from manual/PDF extraction. Must be confirmed before promotion to AcademicTask.
Availability	F1 core	availability_id, owner_id, start_at, end_at, recurrence_type, recurrence_end, timezone, status, version	Represents a free interval. Recurrence end/exception model is unresolved.
BusyInterval / FixedCommitment	F1 required concept; model TBD	commitment_id or source_ref, owner_id, start_at, end_at, label, source, status	Proposal requires checking existing commitments; separate FixedCommitment entity has not been approved. Scheduler requires a unified busy-interval interface regardless of storage choice.
StudyPlan	F1 core	plan_id, owner_id, period_start, period_end, status, version, policy_version, input_snapshot_ref, created_at, approved_at	A plan groups proposed/confirmed blocks. PROPOSED must not be confused with CONFIRMED.
PlannedStudyBlock	F1 core	block_id, plan_id, task_id, start_at, end_at, duration_minutes, status	Specific allocated interval; many blocks may belong to one task and one plan. Cardinality in proposal is conceptual and must be confirmed during schema freeze.
PendingAction	Proposed Orchestrator persistence	pending_action_id, owner_id, intent, slots_json, missing_fields, ambiguities, workflow_state, version, expires_at	Supports multi-turn clarification and approval routing. Storage technology TBD.
ApprovalEvent	Proposed security/audit record	approval_id, owner_id, proposal_id, proposal_version, decision, created_at, expires_at, idempotency_key	Binds decision to exact user/proposal/version. Exact cryptographic/session evidence model TBD.

10.2 Post-MVP entities from proposal

Entity	Fields identified in proposal	Ownership / purpose
LearningMaterial	material_id, source/file reference, topic and source metadata (page/doc references recommended).	Authorized source records used by F2/F3; detailed ingestion model TBD.
QuizAttempt	attempt_id, quiz reference, student responses, submission metadata.	F2 owns attempts; detailed question/response schema TBD.
AssessmentResult	result_id, score, graded_count, incorrect_count and topic summary.	F2 owns objective grading result; deterministic comparison to verified answer keys.
TopicPerformance	topic_id, graded_count, incorrect_count, last_assessed_at, question_refs.	F2 topic-level signal; not a mastery certificate.
RevisionRecommendation	recommendation_id, topic, performance trend, evidence references, rationale.	F2 creates recommendation; F1 requires student confirmation of effort/deadline before schedule proposal.
StudySession	session_id, goal, topic/context and authorized source references.	F3 owns guided learning session context.
SessionOutcome	outcome_id, confirmed summary and source/session references.	F3 saves only after student confirmation; not proof of mastery.

10.3 Relationships and cardinality

Relationship	Proposed cardinality	Notes
Student → AcademicTask	1 to 0..N	Owner isolation required.
Student → Availability	1 to 0..N	A user can declare multiple windows.
AcademicTask → PlannedStudyBlock	1 to 0..N	One task may be split into many blocks; zero blocks allowed.
StudyPlan → PlannedStudyBlock	1 to 0..N	Plan is a container for allocated blocks.
StudyPlan → ApprovalEvent	1 to 0..N	Proposed audit representation; one accepted approval can commit a given version once.
TaskCandidate → AcademicTask	0..1 confirmed task	Promotion only after validation and confirmation.
LearningMaterial ↔ Quiz	many to many (future)	Source grounding/traceability.
QuizAttempt → AssessmentResult	1 to 1 (future proposal)	Each submitted attempt produces one result if validly graded.
StudySession → SessionOutcome	1 to 0..1 (future proposal)	No confirmed outcome if student declines summary.

10.4 Data integrity and lifecycle

- All user-owned records must carry or be query-scoped by owner identity derived from authentication.
- Foreign-key references must not cross owner scope unless an explicit sharing model is approved.
- TaskCandidate values must not be treated as confirmed AcademicTask values.
- PlannedStudyBlock intervals must be valid and linked to one task and one plan.
- A CONFIRMED plan must be committed atomically with its blocks and approval metadata.
- Proposal versions and the input data snapshot/version must be sufficient to detect stale approvals.
- Deletion semantics for tasks with dependent blocks/plans are not specified; UPDATE_TASK and DELETE_TASK remain out of MVP pending decision.

11. TIME AND DATE HANDLING

11.1 Data representation

Value	Recommended representation	Status / rule
Task start/deadline	Date-only ISO 8601: YYYY-MM-DD	PROPOSED canonical form; source uses start date and end date (deadline).
Persisted time interval	Timezone-aware ISO 8601 timestamp / canonical UTC instant plus original timezone metadata	PROPOSED; storage representation TBD.
Timezone	IANA timezone identifier resolved from trusted user profile/configuration	DESIGN; source of profile/configuration and fallback TBD.
Duration	Integer minutes	DESIGN; positive.
Original natural-language phrase	String preserved without normalization loss	DESIGN; keep provenance.

Value	Recommended representation	Status / rule
Recurrence	Explicit recurrence object and timezone	TBD beyond NONE/WEEKLY preliminary modes.

11.2 Natural-language date/time interpretation

- Relative terms (“tomorrow”, “next week”, “this Friday”) must be resolved using request timestamp and trusted timezone; store the resolved value and original phrase.
- Week boundary and “next week” semantics must be defined by one approved locale policy; this is not selected in supplied materials.
- “Morning”, “afternoon”, “evening”, “after class” and similar vague phrases are not exact start times. Ask a clarifying question or show a candidate range for explicit confirmation.
- A duration such as “2 hours” does not define a start time. Ask for a start time if the operation requires one.
- If the task deadline contains only a date, the scheduler must use an approved day-end convention; the exact cutoff is TBD.
- Intervals crossing midnight require a clear date relationship. Do not silently infer a multi-day interval.
- Timezone changes after proposal creation must invalidate or revalidate affected proposals.
- Daylight-saving behavior is relevant only to supported zones that use it; zone database/version handling is TBD.

11.3 Vietnamese examples

User phrase	Safe interpretation	Required system behavior
“Tao rảnh thứ Hai 8h–11h.”	Candidate weekly/date interval only after interpreting the requested Monday.	Resolve date context and timezone; confirm recurrence intent if not explicit.
“Xếp tao 2 tiếng OOP tối mai.”	Intent may be manual block request; duration=120; task ref=OOP; exact start missing.	Resolve date and task; ask start time; do not choose 19:00 automatically.
“19h.” after asking start time	Candidate start time for the uniquely matching PendingAction.	Merge only if one action is waiting for start time; derive end using 120 minutes after confirmation.
“Lập lịch học tuần sau.”	Generate weekly plan; period dates require approved week policy.	Resolve period; use configured timezone; clarify if week policy cannot decide.
“Tối thứ Sáu.”	Ambiguous period, not an exact time.	Ask the student for a start time or choose a candidate explicitly.
“Bài này hạn thứ Sáu.”	Deadline date candidate only.	Resolve which Friday using request date/time; ask if more than one plausible date.

12. ERROR HANDLING AND RECOVERY

Code	Trigger	Owner	Recovery	Write semantics
MISSING_REQUIRED_FIELD	Required candidate/execution field absent.	Orchestrator / validator	Ask for minimum missing information.	No domain write unless draft creation explicitly allows missing field.
INVALID_PARAMETER	Type, enum, range or date relationship invalid.	Validator / backend	Explain field and acceptable correction.	No write.
ENTITY_NOT_FOUND	Task/block/course not found in authenticated owner scope.	Backend	Ask user to search or select again; do not reveal other users’ records.	No write.
AMBIGUOUS_ENTITY	More than one owned entity matches reference.	Orchestrator	Present safe disambiguation choices.	No write.
TIME_CONFLICT	Requested/proposed block overlaps busy/confirmed interval.	Scheduler/backend	Return conflicting interval(s) where safe; propose alternatives.	Do not alter confirmed plan.
INSUFFICIENT_AVAILABILITY	Available capacity less than remaining workload.	Scheduler	Return partial proposal and unscheduled minutes/reasons.	Proposal may be partial; not necessarily operation error.
APPROVAL_REQUIRED	Write/commit lacks valid approval.	Backend	Request explicit student confirmation for exact proposal.	No commit.
STALE_PLAN_VERSION	Proposal or underlying data changed since preview.	Backend	Reject stale approval; refresh/replan.	No stale write.
PROPOSAL_EXPIRED	Proposal passed expiry policy.	Backend/Orchestrator	Create or fetch a current proposal.	No commit.
UNAUTHORIZED	Authenticated actor lacks access to entity/action.	Backend	Return safe generic message.	No data leakage or write.
LLM_INVALID_OUTPUT	Model output malformed or fails extraction schema.	Orchestrator	Bounded retry, then ask user via form/manual input.	No tool dispatch.

Code	Trigger	Owner	Recovery	Write semantics
LLM_TIMEOUT	Model does not return within configured limit.	Orchestrator	Retry under bounded policy or manual fallback.	No duplicate operation.
PERSISTENCE_FAILURE	Database/object store write fails.	Backend	Return failure with correlation ID; retry only if idempotent.	Transaction rollback/consistency procedure.
CONCURRENT_MODIFICATION	Task/availability/ commitment changed while proposal pending.	Backend	Reject and regenerate proposal.	No stale commit.
IDEMPOTENCY_CONFLICT	Same key used with different logical payload.	Backend	Return conflict; do not guess intent.	No duplicate writes.

12.1 Retry policy

Retryable errors must be explicitly classified. The caller must use a bounded retry count and the same idempotency key for the same logical write. Do not retry validation errors, authorization failures, stale approvals or semantic conflicts unchanged. When persistence outcome is unknown after a network interruption, query the operation/idempotency result before attempting another commit.

12.2 Transaction and rollback

A successful approval commit should be atomic for the plan state, block set, approval record and version update. On failure, no partially confirmed plan may be exposed. Technical transaction rollback is a data-consistency mechanism; it is not a user-facing undo feature. Product undo semantics are not specified.

13. SECURITY AND AUTHORIZATION REQUIREMENTS

ID	Security requirement	Priority	Source/status
SEC-001	The backend SHALL derive the acting user identity from an authenticated server-side context, never from LLM output or a client-supplied owner field.	P0	PROPOSED
SEC-002	Every read/write operation SHALL enforce ownership/authorization for each referenced task, block, plan, availability and file.	P0	DESIGN
SEC-003	The system SHALL treat user text, PDF content and LLM output as untrusted data and validate them before use.	P0	DESIGN
SEC-004	The Orchestrator SHALL dispatch only allowlisted operations produced from validated commands.	P0	DESIGN
SEC-005	The system SHALL NOT accept an LLM-generated approval flag as evidence of student approval.	P0	BASELINE / DESIGN
SEC-006	Approval SHALL be bound to authenticated user, exact proposal identifier, exact version and validity window.	P0	DESIGN / PROPOSED
SEC-007	The backend SHALL revalidate ownership, proposal version and current scheduling constraints immediately before commit.	P0	DESIGN
SEC-008	The system SHALL prevent cross-user data disclosure in entity search, conflict messages, logs and errors.	P0	PROPOSED
SEC-009	The system SHOULD record security-relevant audit events for task confirmation, proposal decisions and commits, subject to retention/privacy policy.	P1	PROPOSED / TBD

ID	Security requirement	Priority	Source/status
SEC-010	The system SHALL restrict PDF access and processing to materials authorized for the requesting student and SHALL not expose extracted content outside that scope.	P0	BASELINE / PROPOSED

13.1 Threats and mitigations

Threat	Example	Mitigation / required behavior
Prompt injection in PDF	A document contains text instructing the model to expose data or call tools.	Treat document text as data, not instructions; isolate retrieval/extraction prompt; allowlist tools; validate outputs.
Forged owner/entity ID	LLM returns another user's task_id.	Backend ownership lookup; owner context never comes from model.
Forged approval	LLM emits approval=true or user says "ok" without clear target.	Bind approval to exact proposal/version and authenticated interaction; ask clarification when target ambiguous.
Stale proposal replay	Old approval is submitted after availability changed.	Version/snapshot check; expiry; revalidate constraints; reject stale version.
Duplicate commit/retry	Network timeout causes approval action to be sent twice.	Idempotency key and unique constraint/transaction strategy.
Cross-user data leak	Search result or error reveals another student's task.	Owner filtering at backend query and safe non-disclosing errors.
Unauthorized file reference	User submits file_ref not owned/shared with them.	File access check before parsing and before serving extracted content.

14. NON-FUNCTIONAL REQUIREMENTS

ID	Quality requirement	Priority	Source/status	Verification / measure
NFR-REL-001	The scheduler SHALL enforce availability, commitments, deadline, workload and overlap constraints deterministically.	P0	BASELINE	Constraint violation tests; target violation count = zero for enforced hard constraints.
NFR-REL-002	The backend SHALL preserve confirmed-plan consistency when proposal generation, rejection, cancellation or validation fails.	P0	BASELINE / PROPOSED	Failure injection and state consistency checks.
NFR-REL-003	The system SHALL reject stale approvals when relevant plan or input data has changed.	P0	PROPOSED	Concurrency/stale data tests.
NFR-TRC-001	Extracted task candidates SHALL retain source file/page provenance where available.	P1	PROPOSED	Provenance inspection.
NFR-TRC-002	F2/F3 source-based outputs SHALL retain document/page/topic references in Post-MVP.	P2	BASELINE — POST-MVP	Citation completeness/human review.
NFR-USA-001	The UI SHALL allow students to inspect candidate task details and proposed schedule changes before confirming them.	P0	BASELINE	Usability task walkthrough.
NFR-USA-002	Clarification prompts SHOULD request only the minimum information needed to safely continue.	P1	DESIGN	Clarification evaluation.
NFR-SAFE-001	The system SHALL communicate uncertainty and unsupported evidence rather than presenting it as verified.	P0	BASELINE	Unsupported-source test.

ID	Quality requirement	Priority	Source/status	Verification / measure
NFR-SAFE-002	The system SHALL not represent SessionOutcome or small quiz samples as proof of subject mastery.	P2	BASELINE — POST-MVP	F2/F3 safety tests.
NFR-MNT-001	Feature modules SHALL have separated responsibilities and exchange defined records through the shared orchestration layer.	P1	BASELINE	Architecture/contract review.
NFR-TST-001	The system SHALL expose stable seams for unit, contract, integration and end-to-end testing.	P1	PROPOSED	Testability review.
NFR-PERF-001	The system SHOULD record request latency by orchestration stage and backend operation.	P1	PROPOSED	Telemetry check; numerical target TBD.
NFR-PERF-002	The system SHOULD record PDF parsing and candidate extraction latency separately from user review time.	P1	PROPOSED	Telemetry check; numerical target TBD.
NFR-OBS-001	The system SHOULD associate each request and state transition with a correlation ID.	P1	PROPOSED	Log/trace inspection.
NFR-PRIV-001	The system SHALL apply an approved retention and deletion policy to uploaded documents, extracted text and conversational context.	P0	PROPOSED / TBD	Policy and deletion tests.
NFR-CMP-001	The initial client SHALL be delivered as a web application; exact browser/device compatibility targets remain TBD.	P1	BASELINE / TBD	Compatibility matrix after decision.

No invented performance targets. The supplied proposal and worklist do not approve response-time, throughput, uptime, concurrent-user, extraction-accuracy or model-quality thresholds. This SRS specifies what to measure but leaves numerical acceptance thresholds as TBD pending workload assumptions and benchmark evidence.

Non-functional requirement count: 16. Security requirement count: 10.

15. EXTERNAL AND USER INTERFACE REQUIREMENTS

15.1 Student-facing interface

UI area	Minimum content / behavior	State-aware controls
Dashboard	Upcoming tasks, near deadlines, current week summary and confirmed study plan summary.	Link to task detail/calendar; identify status and time range.
Task entry	Manual form and natural-language entry; show title, course, start date, deadline, priority, estimated effort and missing fields.	Save candidate; edit candidate; confirm; cancel.
PDF import	File selection, processing state, extracted candidate list, source/page references and uncertainty.	Review/edit each candidate; confirm selected candidate(s); manual fallback.
Availability	Date/weekday interval entry, recurrence selection where approved, timezone display.	Preview normalized values; confirm replacement; warn on confirmed-plan conflicts.
Weekly planner	Calendar view with proposed/confirmed distinction, block/task labels, availability and conflicts.	Generate proposal; inspect; approve; reject; request revision.
Rescheduling	Select a confirmed block and specify target date/time/constraint.	Show proposed change and impact; approve/reject.
Errors and recovery	Plain-language error, affected field, safe next action and retry indication.	Never show success when backend write failed or is unknown.

15.2 Existing proposal UI illustrations

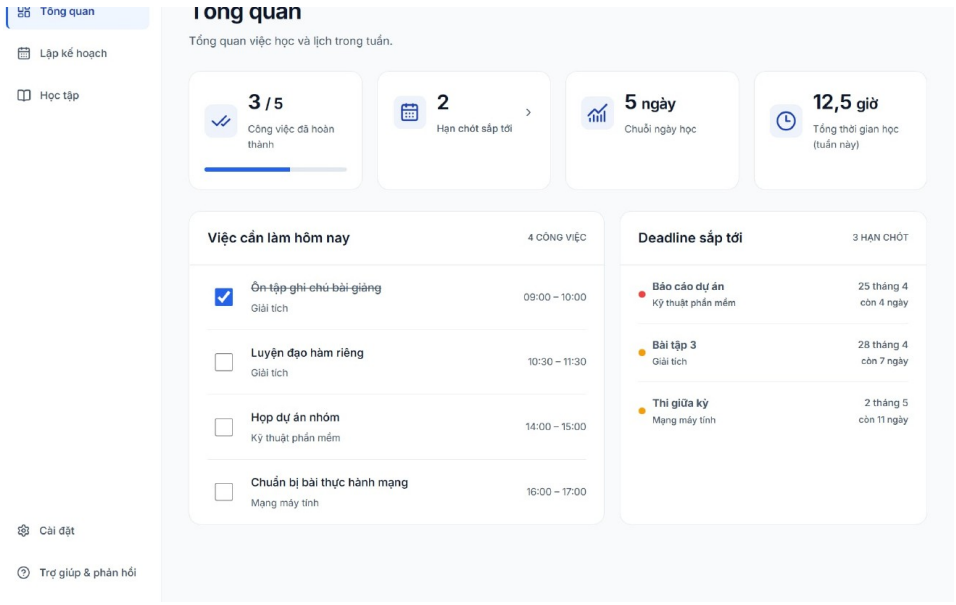


Figure 15-1. F1 dashboard mockup supplied in the updated Midterm Engineering Report. Illustrative design only; not implementation evidence.

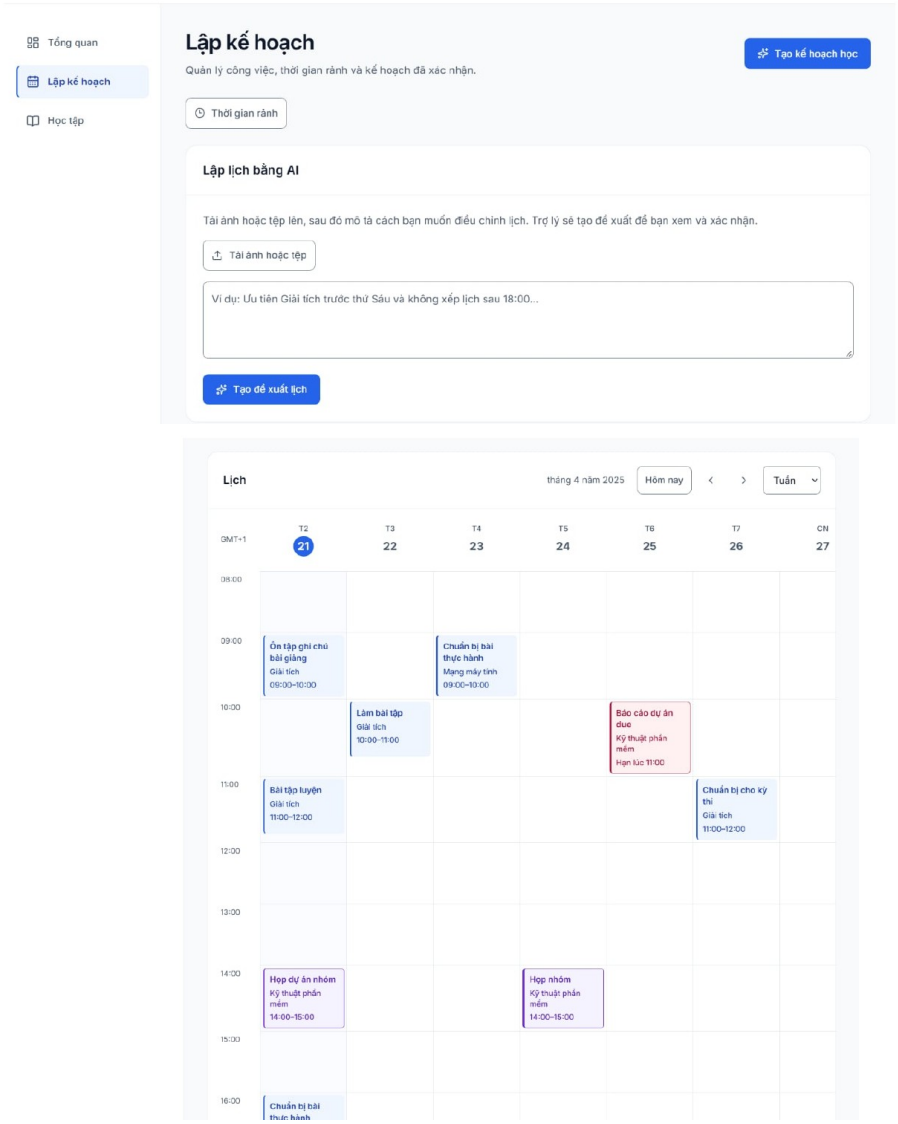


Figure 15-2. F1 task-entry and weekly-calendar mockup supplied in the updated Midterm Engineering Report. Illustrative design only; not implementation evidence.

15.3 Software interfaces

- UI → Orchestrator: user message, attachment references, conversation reference and client action; no trusted user identity in payload.
- Orchestrator → LLM/NLU: constrained extraction/classification request; response validated as untrusted candidate data.
- Orchestrator → F1 backend: validated operation, normalized arguments and trusted identity context.
- F1 backend → academic records: authorized CRUD/query/transaction operations; storage technology TBD.
- F1 backend → UI/Orchestrator: structured ToolResult with status, IDs, versions, unscheduled work and safe error details.
- Future F2/F3 → Shared Orchestrator: typed records and handoff requests; module ownership preserved.

15.4 Communication and protocol decisions

REST/HTTP, WebSocket, server-sent events, RPC, token format, endpoint paths, API versioning strategy, file-upload transport, maximum payload size and CORS/session policy are not specified. The implementation team must select and document these in an architecture decision record before integration. Do not hard-code assumptions into client contracts without a decision.

16. USE CASE SPECIFICATIONS

The following use cases define the principal system interactions. UC-F1-01 to UC-F1-07 belong to the F1 Initial Release. UC-F2-01 and UC-F3-01 document later-release behavior only. The actor is the authenticated University Student unless otherwise stated. All identifiers and API names in this section are logical SRS identifiers, not claims of deployed endpoints.

16.1 UC-F1-01 — Capture, review and confirm an academic task

Field	Specification
Primary actor	University Student
Goal	Create a student-confirmed AcademicTask from manual or natural-language input.
Release	F1 Initial Release / MVP
Preconditions	Student is authenticated; the task form or conversation is available. Course reference may be selected or resolved.
Trigger	Student enters a task or asks the assistant to add one.
Main flow	1. Student submits task information. 2. Orchestrator classifies CREATE_TASK and extracts candidate slots. 3. Validator checks structure, types and relationships. 4. Missing/ambiguous fields are presented for clarification. 5. System shows a reviewable candidate, including normalized dates and estimated effort. 6. Student edits or confirms the candidate. 7. Backend revalidates ownership/context and persists a confirmed task. 8. UI displays the persisted task identifier and confirmed status.
Alternative flows	A. Course name maps to several courses: ask student to select. B. Deadline or effort is missing: preserve a draft candidate if draft policy permits, but do not schedule it. C. Duplicate-looking task: show potential matches and ask whether to create a separate task; duplicate-detection policy is TBD.
Exception flows	Invalid date order, non-positive effort or invalid type: return field-specific error; no confirmed task is written. Persistence failure: do not report success; retain retryable candidate only if its storage succeeded.
Postconditions	Success: one confirmed AcademicTask exists and is owned by the authenticated student. Failure: no confirmed task is created.
Related requirements	F1-TASK-001..010; F1-ORC-001..004; SEC-001..004.
Related tests	TC-01, TC-02, TC-06, TC-07, TC-11, TC-30.

16.2 UC-F1-02 — Import a task candidate from an authorized PDF

Field	Specification
Primary actor	University Student
Goal	Extract candidate task information from a supported text-based PDF and confirm it.
Release	F1 Initial Release / MVP
Preconditions	Student is authenticated; selected file is accessible to that student; accepted file type and size policy is configured.
Trigger	Student uploads or selects an authorized PDF for task capture.

Field	Specification
Main flow	1. UI uploads or references the file. 2. Backend checks authorization and file policy. 3. Parser extracts text from supported text-based pages. 4. AI/NLU proposes task candidates and retains file/page provenance where available. 5. Validator identifies unsupported, missing or ambiguous fields. 6. UI displays candidates and source references. 7. Student selects, edits and confirms a candidate. 8. Backend persists the confirmed task.
Alternative flows	A. Several tasks are detected: show each as a separate candidate for selection/review. B. Several date expressions conflict: ask the student to identify the deadline. C. No task is detected: offer manual entry.
Exception flows	Unauthorized file, encrypted PDF, image-only/unreadable PDF, parser timeout or extraction failure: stop extraction, disclose the reason, offer manual entry; do not invent task data.
Postconditions	Success: confirmed task stored with available provenance. Failure: no confirmed task stored; source file handling follows approved retention policy.
Related requirements	F1-PDF-001..007; F1-TASK-002..006; SEC-010.
Related tests	TC-03, TC-04, TC-05, TC-31.

16.3 UC-F1-03 — Declare or change availability

Field	Specification
Primary actor	University Student
Goal	Record free study intervals that the scheduler may use.
Release	F1 Initial Release; recurrence details require decision.
Preconditions	Student is authenticated; trusted timezone is available or requested from configuration.
Trigger	Student enters one or more free intervals in a form or natural language.
Main flow	1. Student provides date/weekday and start/end time. 2. Orchestrator extracts original expressions. 3. System resolves dates using trusted timestamp/timezone and clarifies ambiguity. 4. Validator checks interval order and recurrence scope. 5. System previews normalized availability and identifies possible impact on confirmed plans. 6. Student confirms a new/replacement interval. 7. Backend persists it and returns the effective availability record.
Alternative flows	A. New availability conflicts with a confirmed plan: explain the conflict and offer a separate replan proposal; do not alter the confirmed plan. B. Weekly recurrence is not approved/configured: accept a date-specific interval or explain the limitation.
Exception flows	Missing exact time, invalid interval or unknown timezone: ask for clarification. Unauthorized or failed persistence: no change is applied.
Postconditions	Availability is available to the scheduler only after successful persistence. Existing confirmed plans remain unchanged unless a separately approved plan is committed.
Related requirements	F1-AVL-001..008; F1-ORC-004; SEC-001..004.
Related tests	TC-08, TC-09, TC-18, TC-19, TC-32.

16.4 UC-F1-04 — Generate, inspect and approve a weekly study plan

Field	Specification
Primary actor	University Student
Goal	Produce a feasible proposed weekly schedule and persist it only after student approval.
Release	F1 Initial Release / MVP — principal use case
Preconditions	Student is authenticated; eligible confirmed tasks and effective availability are available; relevant busy intervals can be queried.
Trigger	Student requests a weekly plan.
Main flow	1. Student states a planning period or uses an approved default. 2. Orchestrator validates request and resolves the period. 3. Backend loads eligible confirmed tasks, remaining workloads, availability, busy intervals, existing confirmed blocks and policy configuration. 4. Deterministic scheduler sorts tasks by deadline and priority tie-break. 5. Scheduler allocates blocks while enforcing hard constraints. 6. Backend returns proposed blocks and explicit unscheduled work. 7. UI displays the proposal, total allocation, affected tasks and constraints. 8. Student approves, rejects or requests revision. 9. For approval, backend verifies authenticated user and exact proposal/version, revalidates underlying data and constraints, and commits atomically. 10. UI displays CONFIRMED only after successful persistence.

Field	Specification
Alternative flows	A. Some work cannot fit: proposal includes partial allocations and unscheduled minutes/reasons. B. Student requests changes: create a new version/proposal; previous version becomes superseded where relevant. C. Student rejects: proposal remains unconfirmed/rejected according to state policy and confirmed plan remains unchanged.
Exception flows	No eligible tasks, no availability, stale inputs, scheduler failure, persistence failure or expired proposal: return accurate status; do not mutate confirmed schedule.
Postconditions	Success: approved plan and blocks are persisted as confirmed. Otherwise: current confirmed plan remains unchanged.
Related requirements	F1-PLAN-001..014; F1-ORC-003..009; SEC-005..007.
Related tests	TC-12..TC-21, TC-25, TC-27, TC-33.

16.5 UC-F1-05 — Retrieve a study plan

Field	Specification
Primary actor	University Student
Goal	Inspect proposed or confirmed blocks in a selected date range.
Release	F1 Initial Release / MVP
Preconditions	Student is authenticated.
Trigger	Student opens the planner or asks to view a schedule.
Main flow	1. Student selects a date range or approved default. 2. Backend queries only the student's authorized plan records. 3. System returns plans and blocks with status, period, version and task references. 4. UI renders calendar entries with proposed/confirmed distinction.
Alternative flows	No plans exist: show empty state and a Generate Plan action. Only a proposal exists: label it clearly as proposed and not confirmed.
Exception flows	Invalid range or retrieval failure: show safe error and do not imply an empty calendar if retrieval failed.
Postconditions	No persistent data is changed.
Related requirements	F1-GET-001..004; SEC-001..004.
Related tests	TC-22, TC-30.

16.6 UC-F1-06 — Propose rescheduling of a confirmed study block

Field	Specification
Primary actor	University Student
Goal	Change the date/time of an existing study block without bypassing approval.
Release	F1 Initial Release / MVP
Preconditions	Student is authenticated; target confirmed block belongs to the student and can be resolved uniquely.
Trigger	Student asks to move a study session/block.
Main flow	1. Orchestrator resolves the target block. 2. If ambiguous, student selects one. 3. System collects new time/constraints and retains current duration if appropriate. 4. Backend validates availability, commitments, task deadline, remaining effort and overlaps. 5. Backend creates a revised proposal referencing the current block and source version. 6. UI displays old and proposed times plus consequences. 7. Student approves or rejects. 8. On approval, backend revalidates and commits using concurrency/idempotency controls.
Alternative flows	Student asks for a different time after seeing proposal: create a new proposal version and invalidate the old approval context. Student rejects: original confirmed block remains.
Exception flows	Block not found, unauthorized block, stale source data, conflict or expired proposal: return appropriate error and do not alter the existing block.
Postconditions	Success: new block state committed. Failure/rejection: original confirmed block remains effective.
Related requirements	F1-RSCH-001..006; F1-ORC-006..009; SEC-006..007.
Related tests	TC-23..TC-25, TC-27, TC-28.

16.7 UC-F1-07 — Resolve clarification and pending action across turns

Field	Specification
Primary actor	University Student
Goal	Complete an incomplete request through multi-turn conversation without misrouting the response.
Release	F1 Initial Release / MVP
Preconditions	A PendingAction exists or a new request has an unresolved field.
Trigger	Student answers a clarification, corrects a value, cancels, or changes topic.
Main flow	1. System asks a focused question for the blocking field. 2. PendingAction stores intent, slots, missing fields, version and expiry. 3. Student replies (e.g., “19h”). 4. Orchestrator determines whether exactly one pending action matches. 5. Answer is merged with provenance. 6. Validation runs again. 7. System continues, asks a follow-up, or routes the validated command.
Alternative flows	Several pending actions could match: ask which request the answer belongs to. User corrects a prior slot: update pending version and revalidate. User changes topic: suspend, cancel or retain previous action only according to policy; do not apply unrelated text to it.
Exception flows	Pending action expired or proposal superseded: explain and restart/retrieve current request. User says “đồng ý” with no unique active proposal: ask which proposal, or decline to commit.
Postconditions	State reflects the exact request/decision; no unintended side effect occurs.
Related requirements	F1-ORC-001..009; SYS-FR-004..005.
Related tests	TC-06..TC-10, TC-26..TC-29.

16.8 UC-F2-01 — Complete a verified quiz and review revision priorities (Post-MVP)

Field	Specification
Primary actor	University Student
Preconditions	Authorized course material and questions with reviewed, source-supported answer keys are available for the selected topic.
Main flow	Student selects topic → system displays eligible verified MCQs → student submits responses → deterministic grading compares responses to verified keys → system records QuizAttempt/AssessmentResult → topic metrics are calculated → revision recommendations are presented with evidence.
Alternatives/exceptions	No verified questions or source support: explain limitation and do not grade unsupported questions. No valid responses: mark INSUFFICIENT_DATA. Small sample: do not claim mastery. Schedule update: F1 separately confirms effort/deadline, validates feasibility, proposes and asks student approval.
Postconditions	Assessment result and supported recommendation are recorded; no confirmed schedule changes without F1 approval.
Scope note	Post-MVP only; question formats beyond verified multiple-choice, essay grading and predictive mastery modeling are excluded by proposal.

16.9 UC-F3-01 — Complete a source-grounded guided learning session (Post-MVP)

Field	Specification
Primary actor	University Student
Preconditions	Authorized relevant course material is available. An approved F1 block may optionally supply context.
Main flow	Student starts session and states goal/question → F3 retrieves relevant passages and metadata → F3 provides explanation/progressive hint grounded in retrieved evidence → student follows up or ends session → system presents summary → student reviews and confirms → F3 saves SessionOutcome.
Alternatives/exceptions	Retrieved material does not support answer: state limitation and ask clarification/additional material. Student declines summary: do not save as confirmed outcome. Optional quiz handoff: route to F2 only on request/approved flow.
Postconditions	Confirmed SessionOutcome exists only after student confirmation. It is not an AssessmentResult or proof of mastery.
Scope note	Post-MVP only; voice/video tutoring and unsupported general-purpose answers are out of scope.

17. ACCEPTANCE CRITERIA

Acceptance criteria below define observable release checks. They are proposed verification criteria derived from the proposal and design. Numerical performance thresholds remain TBD; hard scheduling invariants are not relaxed by that omission.

ID	Acceptance criterion	Related requirements	Evidence required
AC-001	A student can manually enter a task and see a candidate before confirmation.	F1-TASK-001..003	UI/E2E recording and persisted state
AC-002	A candidate with missing deadline or effort is not eligible for automatic scheduling.	F1-TASK-002, 005, 010	DB fixture and scheduler test
AC-003	A supported text PDF produces reviewable task candidates with file/page provenance where available.	F1-PDF-001..005	Parser output, source references and review trace
AC-004	An unreadable/image-only PDF triggers a manual-entry fallback and no fabricated confirmed task.	F1-PDF-006	E2E evidence and database assertion
AC-005	A confirmed task cannot be scheduled by another user's request.	F1-TASK-008; SEC-001..002	Cross-user authorization test
AC-006	Invalid/non-positive estimated effort is rejected before scheduling eligibility.	F1-TASK-005..006	Unit test report
AC-007	Ambiguous task references require explicit selection before a write/proposal.	F1-TASK-009; F1-RSCH-002	Conversation transcript and dispatch log
AC-008	Availability values are normalized using trusted timezone and invalid intervals are rejected.	F1-AVL-002..003	Time parser/validator tests
AC-009	A change to availability that conflicts with a confirmed plan does not silently change that plan.	F1-AVL-005, 007	Before/after database comparison
AC-010	Scheduler uses deadline-first order and priority only as the documented equal-deadline tie-break.	F1-PLAN-003	Deterministic fixture with tied deadlines
AC-011	Repeated scheduling with identical inputs, data snapshot and policy version yields identical output.	F1-PLAN-004, 014	Repeatability test
AC-012	Every allocated block falls within the valid planning period and availability and respects task deadline and busy intervals.	F1-PLAN-005..009	Constraint validator report
AC-013	No two blocks overlap under half-open interval semantics.	F1-PLAN-007	Boundary and pairwise overlap tests
AC-014	Insufficient capacity produces explicit unscheduled minutes/reasons; it does not fabricate a fully scheduled result.	F1-PLAN-010	Partial-plan fixture
AC-015	A generated plan is displayed as PROPOSED until approval and successful commit.	F1-PLAN-011..013	UI plus persistence state trace
AC-016	Rejecting, cancelling, expiring or failing a proposal leaves the existing confirmed schedule unchanged.	F1-PLAN-013; F1-RSCH-005	State/persistence regression test
AC-017	A reschedule proposal identifies its source block and validates all applicable hard constraints.	F1-RSCH-001..004	Proposal payload and scheduler tests
AC-018	Approval for a wrong user, proposal, version or expired context is rejected.	F1-ORC-008; SEC-005..007	Security/concurrency tests
AC-019	Duplicate approval/retry does not create duplicate commits or duplicate blocks.	SEC-007; proposed idempotency rules	Replay test and unique persistence evidence
AC-020	The UI reports success only after confirmed persistence; failure/unknown result is never shown as committed.	F1-ORC-009; NFR-REL-002	Failure-injection test

18. TESTING AND QUALITY ASSURANCE REQUIREMENTS

18.1 Test levels

Test level	Purpose	Primary scope	Required evidence
Unit	Verify pure validators, interval operations, sorting, allocation and status transitions.	Domain and scheduler	Repeatable automated test output
Contract	Verify schemas, nullability, enums, request/result envelopes and version semantics.	LLM/Orchestrator/backend boundaries	Schema validation and compatibility report
Integration	Verify authorization, persistence, file parsing, orchestrator dispatch and backend interactions.	Component boundaries	Integration logs and database assertions
End-to-end	Verify the complete student journey from message/form to confirmed persisted plan.	F1 MVP	Reproducible scripted user journeys
Security	Verify cross-user isolation, forged IDs, prompt injection, forged approval and replay protection.	All trust boundaries	Security test results
Regression	Ensure changes do not violate already approved requirements or alter confirmed schedules unexpectedly.	F1 workflows	Versioned regression suite
AI/NLU evaluation	Measure intent/slot extraction and candidate quality separately from deterministic business correctness.	Extraction and clarification only	Frozen test set, annotations and metric report

The test set SHALL separate development examples from a held-out evaluation set before comparing candidate models. Model selection must consider multi-intent handling, slot extraction, clarification, dispatch correctness and end-to-end behavior—not intent accuracy alone. No benchmark result is claimed by this SRS.

18.2 Test scenario matrix

The following 34 scenarios are the minimum initial matrix. Each test must include the user input, trusted request context, database fixtures, initial state, expected intent/slots/status/tool/state/response and allowed/forbidden side effects.

Test ID	Scenario / input	Expected result	Forbidden side effect
TC-01	Complete manual task entry with title, course, start/deadline, priority and effort.	Candidate shown; after student confirmation, confirmed task stored.	Scheduling an unconfirmed candidate.
TC-02	Task has missing deadline or effort.	Missing field identified; task remains non-schedulable.	Inventing a date/effort or scheduling.
TC-03	Upload authorized text-based PDF with a recognizable assignment.	Extract candidate fields and source page; student review required.	Auto-confirming extracted values.
TC-04	PDF has multiple task candidates.	Display separate candidates and request selection/review.	Combining unrelated tasks silently.
TC-05	PDF is image-only, encrypted or unreadable.	Explain unsupported extraction and offer manual entry.	Pretending OCR/text extraction succeeded.
TC-06	User says “Thêm bài OOP”, no remaining details.	Ask only fields needed by draft/confirmation; preserve PendingAction.	Creating confirmed task with fabricated attributes.
TC-07	User enters negative or non-numeric effort.	Structural/semantic validation error.	Persisting invalid workload.
TC-08	User says “tối thứ Hai” without exact time.	Ask for an exact interval or approved configured default if one exists.	Choosing arbitrary 19:00.
TC-09	User says “rảnh thứ Hai 8–11h”.	Resolve date/weekday and timezone; validate interval.	Treating availability as a study task or schedule change.
TC-10	Two tasks/blocks share similar names.	Show candidates and require explicit selection.	Guessing an entity ID.
TC-11	User references an entity owned by another student.	Non-disclosing authorization/not-found response.	Revealing entity existence or details.
TC-12	Generate plan with complete eligible tasks and availability.	Deterministic proposed plan returned.	Directly committing confirmed schedule.

Test ID	Scenario / input	Expected result	Forbidden side effect
TC-13	Distinguish “lập lịch tuần sau”, “xếp buổi OOP 8–10h”, “tôi rảnh 8–10h”.	Map respectively to GENERATE_STUDY_PLAN, scope-dependent CREATE_STUDY_BLOCK, SET_AVAILABILITY.	Dispatching manual block when out of scope.
TC-14	Two tasks share deadline but have different priorities.	Higher priority is ordered first per approved priority convention.	Non-deterministic tie handling.
TC-15	Busy commitment overlaps an availability interval.	Busy interval excluded from scheduling capacity.	Allocating a block inside busy time.
TC-16	Task deadline lies inside requested planning interval.	No proposed block extends beyond deadline boundary.	Scheduling past deadline.
TC-17	Total remaining work exceeds free time.	Partial proposal with unscheduled minutes/reasons.	Claiming all work is scheduled.
TC-18	Availability is changed so it conflicts with confirmed blocks.	Warn; require replan proposal and approval.	Mutating confirmed blocks automatically.
TC-19	Availability replacement would remove existing intervals.	Preview and explicit confirmation required.	Overwriting availability silently.
TC-20	Student views generated plan before approval.	State is PROPOSED, not CONFIRMED.	Displaying proposal as persisted confirmed calendar.
TC-21	Student rejects, cancels or lets plan proposal expire.	Current confirmed plan unchanged.	Partial write to confirmed schedule.
TC-22	Retrieve a range with no plans versus retrieval backend failure.	Empty state only for successful empty query; safe error on failure.	Confusing failure with empty result.
TC-23	Reschedule a block to a conflicting time.	Reject or return alternative proposal only; original block unchanged.	Updating target block directly.
TC-24	Student requests reschedule with only new date/time.	Preserve known duration and propose valid new interval.	Resetting duration arbitrarily.
TC-25	Availability/task/commitment changes after proposal creation.	Stale proposal rejected; refresh/replan required.	Committing stale proposal.
TC-26	Student replies “19h” to one active clarification.	Merge into the correct PendingAction and revalidate.	Starting unrelated intent.
TC-27	Student says “đồng ý” while two proposals are pending.	Ask which proposal; no commit yet.	Approving both or guessing.
TC-28	Student changes a slot after proposal preview.	New version/proposal required; old approval context invalidated.	Reusing old approval for changed proposal.
TC-29	Pending action expires or student changes topic.	Apply documented expiry/topic policy; ask to resume if necessary.	Merging unrelated follow-up into expired action.
TC-30	Retry after backend timeout where commit outcome is unknown.	Query commit/idempotency state; report accurately.	Blind duplicate commit or false success.
TC-31	PDF contains prompt-injection instructions.	Treat contents as document data; ignore instructions; no unauthorized tool call.	Following document instructions as system commands.
TC-32	Two availability intervals touch at boundary (e.g., 08:00–09:00 and 09:00–10:00).	Half-open interval semantics treat them as adjacent, not overlapping.	False positive overlap.
TC-33	Two users approve proposals concurrently or same proposal is approved twice.	At most one valid commit; subsequent attempt is idempotent/stale.	Duplicate blocks or lost update.
TC-34	LLM timeout, malformed JSON or unsupported intent.	Bounded retry/fallback/clarification; no backend write.	Dispatching malformed or unsupported command.

18.3 AI/NLU metrics

Metric	Definition / method	Note
Intent precision/recall/F1	Compare predicted intent labels with human-annotated held-out examples.	Report per intent and macro average; multi-label cases evaluated separately.
Slot exact match	Fraction of required slots with exact normalized match.	Keep raw expression and normalization accuracy separate.
Slot field accuracy	Per-field precision/recall or exact match for dates, effort, task references and times.	Date/time scoring uses a frozen timezone and locale.
Clarification appropriateness	Human/criteria-based assessment of whether the system asks for a genuinely blocking field and does not ask for known context again.	Define rubric before benchmark.
Dispatch correctness	Fraction of requests dispatched to the correct allowlisted operation after validation.	Include confusion between availability, automatic planning and manual block.
End-to-end completion	Fraction of valid scripted journeys reaching the intended safe terminal state.	Separate completion from user approval rate.

Metric	Definition / method	Note
Constraint violation rate	Invalid proposed blocks / all proposed blocks, categorized by constraint.	For hard constraints, release expectation is zero on deterministic test suite.
Unauthorized write rate	Writes performed without valid identity/authorization/approval.	Must be zero in security tests.
Latency	Record stage and end-to-end latency distributions.	Numerical SLO/target is TBD.

18.4 Test data governance

Use synthetic or permissioned task/availability records for development and security tests. Keep model-development samples separate from the held-out set. Store no real student content in logs unless approved by a documented privacy policy. Freeze timezone, policy version, and expected outputs for deterministic scheduler fixtures. Annotators and evaluation rules are TBD until assigned by the team.

19. REQUIREMENT TRACEABILITY MATRIX

The matrix below traces every requirement family and every individual identifier through source, objective, component, use case and verification. Requirement IDs not listed in a family range are not implied; each row lists the complete IDs covered.

Requirement IDs	Source / classification	Objective and component	Use case	Verification
SYS-FR-001..006	Updated proposal §§1.4, 3.1; context §§4, 9; BASELINE/DESIGN/PROPOSED	Student web workflow; UI/Orchestrator	UC-F1-01..07	Contract, UI/E2E, status tests
F1-TASK-001..010	Proposal §§2.1.1, 2.3.2, 3.2; context §§5–8; BASELINE/DESIGN/PROPOSED	Capture, confirm and resolve tasks; UI/Orchestrator/F1 backend	UC-F1-01	TC-01, TC-02, TC-06, TC-07, TC-10, TC-11
F1-PDF-001..007	Proposal §§1.4, 2.1.1, 4.4; BASELINE/PROPOSED/TBD	Authorized text PDF extraction; UI/parser/F1 backend	UC-F1-02	TC-03..TC-05, TC-31
F1-AVL-001..008	Context §§6–8; worklist P0-03/P0-05; DESIGN/PROPOSED/TBD	Time availability and conflict management; Orchestrator/F1 backend	UC-F1-03	TC-08, TC-09, TC-18, TC-19, TC-32
F1-PLAN-001..014	Proposal §§2.1.1, 2.3.2, 3.2.4; context §§5, 8; BASELINE/DESIGN/PROPOSED/TBD	Feasible deterministic weekly planning; F1 scheduler/backend	UC-F1-04	TC-12..TC-21, TC-14..TC-17, TC-33
F1-GET-001..004	Context §§6, 8; proposal UI mockups; DESIGN/PROPOSED	Inspect proposed/confirmed plan; UI/F1 backend	UC-F1-05	TC-22
F1-RSCH-001..006	Proposal §§1.4, 2.1.1, 2.3.2, 3.2.4; context §§8–9; BASELINE/DESIGN/PROPOSED	Student-approved block change; Orchestrator/F1 backend	UC-F1-06	TC-23..TC-25, TC-28, TC-33
F1-ORC-001..009	Context §§4, 7–9; worklist P0-02/P0-04/P0-06; DESIGN/PROPOSED	Safe conversational command routing; Shared Orchestrator	UC-F1-01..07	TC-06..TC-10, TC-26..TC-30, TC-34
F2-FR-001..005	Proposal §§2.1.2, 2.3.2, 3.1–3.2; BASELINE — POST-MVP	Verified quiz and topic-level assessment; F2 module	UC-F2-01	Post-MVP source/grading tests; not F1 MVP gate
F3-FR-001..005	Proposal §§2.1.3, 2.3.2, 3.1–3.2; BASELINE — POST-MVP	Source-grounded learning session; F3 module	UC-F3-01	Post-MVP grounding/session tests; not F1 MVP gate
SEC-001..010	Proposal §§1.2, 2.1.1, 3.2; context §§4, 8; BASELINE/DESIGN/PROPOSED/TBD	Identity, ownership, safe approval and files; Orchestrator/backend	UC-F1-01..07	TC-11, TC-18, TC-25, TC-27, TC-30, TC-31, TC-33
NFR-REL-001..003	Proposal §§2.1.1, 2.2; context §§8–9; BASELINE/PROPOSED	Determinism and state consistency; scheduler/backend	UC-F1-03..06	TC-12..TC-25, TC-30, TC-33
NFR-TRC-001..002	Proposal §§2.1.1–2.1.3; BASELINE/PROPOSED	Provenance; PDF parser/F2/F3	UC-F1-02, UC-F2-01, UC-F3-01	TC-03; post-MVP traceability tests
NFR-USA-001..002	Proposal §§1.2, 2.1.1, 3.3; BASELINE/PROPOSED	Review/clarification usability; UI/Orchestrator	UC-F1-01..07	Usability walkthrough
NFR-SAFE-001..002	Proposal §§2.1.2–2.1.3, 4.4; BASELINE	Evidence/uncertainty handling; F2/F3 and shared layer	UC-F2-01, UC-F3-01	TC-05, TC-31; Post-MVP tests

Requirement IDs	Source / classification	Objective and component	Use case	Verification
NFR-MNT-001	Proposal §§3.1–3.2; BASELINE	Separated feature modules; architecture	All feature use cases	Architecture and contract review
NFR-TST-001	Worklist P0-07/P1-04; PROPOSED	Testable contracts and domain seams	UC-F1-01..07	Testability review
NFR-PERF-001..002	Worklist P1-01/P0-07; PROPOSED/TBD	Observability and benchmark; Orchestrator/parser/backend	UC-F1-01..07	Latency telemetry; targets TBD
NFR-OBS-001	Worklist P0-06; PROPOSED	Correlation and auditability; all backend operations	UC-F1-01..07	Trace/log inspection
NFR-PRIV-001	Proposal §1.4, worklist P0-06; PROPOSED/TBD	Data lifecycle; file store/context store	UC-F1-02, UC-F1-07	Deletion/retention tests after policy decision
NFR-CMP-001	Proposal §1.4; BASELINE/TBD	Web client compatibility; UI	UC-F1-01..07	Browser/device matrix after decision

19.1 Traceability maintenance rule

When a requirement is added, changed, deferred or removed, the change SHALL update its source/status, affected use cases, tool/data contracts, acceptance criteria and test cases in the same change set. A requirement without an identified verification method is not ready for implementation sign-off. For the source-to-objective mapping: the principal objective is a student-controlled, feasible F1 weekly plan; PDF capture is an input route, not a separate scheduling engine; F2/F3 remain downstream planned capabilities.

20. OPEN DECISIONS, RISKS AND REQUIRED TEAM ACTIONS

The following items are intentionally left open because the supplied sources do not show a team decision. “Suggested resolution” is advice, not a decision. Priorities follow the supplied worklist: P0 design freeze, P1 implementation dependency, P2 later scope.

ID	Priority	Open issue / risk	Why it matters	Decision or evidence required	Suggested next step (not approved)
OD-001	P0	Confirm the exact F1 Initial Release baseline against the updated proposal and any instructor feedback.	Prevents scope drift.	Team-approved in/out list.	Freeze task capture, text-PDF extraction, weekly plan, view and approved reschedule as F1 scope.
OD-002	P0	Decide whether UPDATE_TASK is in MVP.	Task edits can invalidate plans/blocks.	Scope decision and impact policy.	Defer unless required; if included, define proposal/approval impact.
OD-003	P0	Decide whether DELETE_TASK is in MVP.	Deletion may orphan blocks or invalidate plan.	Scope decision and referential behavior.	Defer unless required; define archive/delete and dependent-record behavior.
OD-004	P0	Decide whether manual CREATE_STUDY_BLOCK is in MVP.	Distinct from automatic planning and availability.	Scope decision and UX/use case.	Keep deferred unless proposal/instructor requires it.
OD-005	P0	Decide how busy commitments are represented and whether FixedCommitment is a separate entity.	Scheduler must avoid unavailable intervals even if entity design is open.	Data model / ownership / recurrence decision.	Implement a stable busy-interval interface first; decide physical entity later.
OD-006	P0	Freeze Orchestrator Input/Output Contract.	All components need stable boundaries.	Schema for UserMessage, ExtractionResult, PendingAction, ValidatedCommand, ToolResult and approval context.	Complete P0-02 with machine-valid schemas and invalid examples.
OD-007	P0	Canonical field vocabulary and semantics.	Inconsistent names cause integration failures.	Confirm task_candidate_id, slots, deadline_date, duration_minutes, start_at/end_at.	Adopt a single glossary/schema and migrate examples.

ID	Priority	Open issue / risk	Why it matters	Decision or evidence required	Suggested next step (not approved)
OD-008	P0	Complete tool catalog and intent-to-operation allowlist.	Prevents arbitrary LLM tool calls and missing workflow branches.	Operation contracts, side effects, errors, authorization and confirmation.	Complete P0-03; mark unsupported operations explicitly.
OD-009	P0	PendingAction lifecycle and routing policy.	Short replies can attach to the wrong request.	Multiple pending actions, correction, cancellation, expiry, topic change and version semantics.	Build transition table and test multi-turn cases.
OD-010	P0	Multi-intent sequencing and dependency policy.	Availability may need confirmation before plan generation.	Supported combinations, ordering and partial success semantics.	Support only small deterministic sequential compositions for MVP.
OD-011	P0	Timezone and date/time normalization.	Incorrect date parsing can schedule outside the intended period.	Trusted timezone source, week start, relative-date reference, date-only deadline boundary, overnight interval behavior.	Use configured timezone and preserve original expressions; approve parsing conventions.
OD-012	P0	Scheduler tie-breaks, block splitting, workload and remaining-effort calculations.	Same data must yield feasible and repeatable plans.	Minimum/maximum block size, granularity, daily workload caps, break rules, equal-deadline/equal-priority tie-breaks.	Put policy constants in versioned configuration; benchmark before fixing defaults.
OD-013	P0	Partial proposal vs INSUFFICIENT_AVAILABILITY error semantics.	UI must distinguish a useful partial plan from total failure.	When partial proposal is valid; how minutes/reasons are represented.	Return proposal with unscheduled_work if at least one block is feasible; use error when none can be proposed, subject to team approval.
OD-014	P0	Approval evidence, proposal expiry and version binding.	Prevents forged/stale approvals.	UI/chat approval mechanism, expiry policy and authenticated interaction evidence.	Bind approval to user + proposal ID + version + expiry; never rely on LLM boolean.
OD-015	P0	Concurrency, transaction boundary, idempotency and recovery.	Retry/double approval must not duplicate or overwrite data.	Storage-level optimistic concurrency/locking and idempotency strategy.	Decide atomic commit boundary and test replay/concurrent approval.
OD-016	P0	Acceptance thresholds and evaluation dataset.	Team cannot claim quality without defined evidence.	Annotated Vietnamese dataset, metrics and release threshold.	Freeze held-out test set before model comparison; no threshold assumed here.
OD-017	P1	Backend/API framework, database and deployment architecture.	Affects implementation structure and interfaces.	Team skill, runtime, hosting, operations and course constraints.	Compare candidates through ADRs; do not infer from proposal.
OD-018	P1	LLM/NLU model, router and inference provider.	Cost, latency, privacy and extraction accuracy.	Candidate model/version, license, data policy and end-to-end benchmark.	Benchmark multiple candidates against TC matrix; JEV router and Qwen remain candidates, not choices.
OD-019	P1	File limits and retention/deletion policy.	Privacy, storage cost and parse reliability.	Maximum size/pages, allowed encryption, retention, deletion and access.	Configure limits and retention; document student-facing disclosure.
OD-020	P1	Browser/device compatibility and non-functional service targets.	Needed for release-level test plan.	Supported browsers, viewport/device scope, latency/availability/concurrency targets.	Gather workload assumptions and measure prototype before setting numeric SLOs.

ID	Priority	Open issue / risk	Why it matters	Decision or evidence required	Suggested next step (not approved)
OD-021	P1	Team ownership, review schedule and completion dates.	No execution commitment is evidenced in source worklist.	Team agreement assigning owners/dates.	Assign only in the team's project tracker; SRS intentionally leaves TBD.

20.1 Risk register

Risk ID	Risk	Likelihood / impact	Mitigation	Residual issue
R-001	Incorrect extraction produces a wrong task/deadline.	Medium / High	Candidate-only extraction, provenance, student review, field validation.	Extraction quality needs benchmark.
R-002	Insufficient availability makes full schedule infeasible.	High / Medium	Partial scheduling and explicit unscheduled minutes; never fabricate capacity.	Partial-vs-error threshold needs decision.
R-003	Ambiguous dates or timezone produce incorrect calendar placement.	Medium / High	Trusted timestamp/timezone, original expression retention, clarification.	Week boundary/date-only policy TBD.
R-004	Changed source data invalidates proposal.	Medium / High	Version/fingerprint and backend revalidation on commit.	Data snapshot strategy TBD.
R-005	Duplicate approval or retry creates duplicate writes.	Medium / High	Transaction, idempotency and concurrency checks.	Persistence-specific design TBD.
R-006	Prompt injection in uploaded PDF manipulates AI behavior.	Medium / High	Treat file text as untrusted content; tool allowlist and backend enforcement.	Security review required.
R-007	Scope creep delays F1 validation.	High / Medium	Keep F2/F3 and P2 capabilities outside MVP.	Scope approval required.
R-008	Model selected from intent accuracy only fails end-to-end.	Medium / High	Evaluate slots, clarification, dispatch, constraint and full workflows.	Evaluation set and threshold TBD.
R-009	Unclear file retention creates privacy or storage risk.	Medium / High	Retention configuration, access control, deletion process.	Policy approval required.
R-010	F2/F3 ownership is coupled to F1 internals.	Medium / Medium	Separate modules, shared typed records and orchestrator handoffs.	Future contract evolution required.

21. IMPLEMENTATION ROADMAP AND RELEASE GATES

This roadmap reflects the sequence in cong_viec_can_lam.md; it is an ordering proposal, not an approved calendar. No owners or completion dates are assigned here.

21.1 P0 — Freeze requirements and design contracts

Work item	Required output	Dependency / exit gate
P0-01 Scope/baseline	F1 in-scope/deferred list, proposal alignment and instructor/team questions.	Exit when no deferred feature is silently mandatory.
P0-02 Orchestrator contract	Machine-valid I/O schemas, provenance and approval context.	Can begin immediately; exit when all component boundaries are explicit.
P0-03 Tool contracts	Complete tool catalog, dispatch allowlist and intent-operation mapping.	Depends on P0-01/P0-02.
P0-04 State/routing	Workflow transition table and multi-turn scenarios.	Depends on P0-02/P0-03.
P0-05 Time/scheduler rules	Time normalization, entity resolution and deterministic scheduler rule sheet.	Depends on P0-01/P0-02; used by P0-03/P0-04.
P0-06 Approval/recovery	Commit sequence, version/concurrency/idempotency and failure policy.	Depends on P0-03/P0-04/P0-05.
P0-07 Test matrix	Vietnamese test data, expected outcomes, metrics and approved thresholds.	Draft alongside P0-02; finalize after P0-03..P0-06.

P0 exit gate: The team approves scope, schemas, scheduler invariants, state transitions, approval semantics, error/recovery rules and the acceptance/test plan. Any unapproved item remains visibly TBD and cannot silently become implementation behavior.

21.2 P1 — Implement and verify F1 MVP

1. Select backend/API/database/model options with documented architecture decisions and benchmark evidence (P1-01).
2. Implement F1 domain and persistence: task candidate/confirmation, availability, commitments/busy intervals, plan/block/version and validators (P1-02).
3. Implement deterministic scheduler with hard-constraint checks, partial/unscheduled output, and proposal lifecycle.
4. Implement approval/commit with backend revalidation, transaction, optimistic concurrency and idempotency.
5. Implement Shared Orchestrator: NLU adapter, schema validation, entity resolution, PendingAction store, clarification, allowlist dispatch, approval routing and recovery (P1-03).
6. Implement minimum UI: task entry/PDF import, availability, weekly planner, proposal preview, approve/reject and errors.
7. Run test matrix, integration/E2E/security checks and model evaluation; record environment, dataset, metrics, failures and limitations (P1-04).

P1 release gate: Demonstrate actual running implementation for the agreed F1 scope; pass agreed hard-constraint/security invariants; complete the relevant acceptance tests; show persisted approved plans and correct rejection/stale/retry behavior. A mockup, generated JSON or happy-path demo alone is insufficient evidence of MVP completion.

21.3 P2 and Post-MVP

- P2: UPDATE_TASK, DELETE_TASK, manual CREATE_STUDY_BLOCK, separate FixedCommitment, advanced recurrence and optional routing optimization only after scope approval.
- Post-MVP F2: authorized source retrieval, question verification, deterministic MCQ grading, topic-level performance and revision recommendations.
- Post-MVP F3: authorized source-grounded explanations/hints, session lifecycle, summary review and confirmed SessionOutcome.
- Cross-feature integration: only typed, permission-checked handoffs; F2/F3 outputs must not modify confirmed F1 schedules without student confirmation and F1 validation/approval.

21.4 Team roles recorded by the proposal

The updated proposal records planned responsibilities as follows. These are proposal-level role assignments, not evidence of completed implementation or fixed task ownership for every SRS work item.

Member	Proposal role	Responsibility recorded in proposal
Châu Hoà Phát	Project Manager; Backend Developer	Coordinate progress and backend; lead slide preparation with support from Hùng.
Phạm Hùng Khôi	Full-Stack Developer	Develop/integrate frontend and backend workflows.
Nguyễn Ngọc Thanh Hương	UI/UX Designer; Frontend Developer	Lead proposal preparation and coordinate task assignments.
Nguyễn Trần Minh Khang	UI/UX Designer; Frontend Developer	Design interfaces and implement frontend components.
Huỳnh Nguyễn Thế Hiển	Backend Developer; AI Engineer	Develop backend services and AI integration.
Đặng Minh Hùng	Backend Developer	Develop backend services and support slide preparation.

The actual owner of each P0/P1 work item, reviewer and due date remains TBD and must be approved by the team.

APPENDICES

Appendix A — Canonical terminology proposed for implementation

Term	Meaning	Important distinction
AcademicTask	A unit of academic work with an effort estimate and deadline.	Not a calendar interval.

Term	Meaning	Important distinction
TaskCandidate	Untrusted/manual/AI-extracted task information awaiting review/confirmation.	Not schedulable.
PlannedStudyBlock	A concrete time interval allocated to an AcademicTask.	One task may have multiple blocks.
StudyPlan	A versioned collection of proposed or confirmed blocks for a planning period.	Not equivalent to one block.
Availability	Student-declared free time.	Not proof that no other commitment exists.
BusyInterval	Any interval that must not be scheduled over.	May be sourced from a FixedCommitment or another approved record; entity choice TBD.
Proposal	A proposed change awaiting student decision.	Not committed.
Approval	Authenticated student decision bound to a target proposal/version.	Not an LLM extraction field.
Commit	Successful backend persistence after revalidation.	Not the same as READY or approval receipt.
PendingAction	Versioned conversational workflow awaiting input or decision.	Not a domain entity such as Task or StudyPlan.
ExtractionResult	Untrusted output of the NLU/LLM layer.	Not an executable command.
ValidatedCommand	Normalized operation after structural, semantic, domain and authorization checks.	Still does not replace required approval.
SessionOutcome	Student-confirmed summary of a learning session (F3).	Not proof of mastery or a graded assessment.
AssessmentResult	Deterministic result from verified quiz answer keys (F2).	Not interchangeable with SessionOutcome.

Appendix B — Illustrative contract examples

The following examples illustrate the proposed logical contracts. They are not deployed API payloads or finalized schemas. Fields and enum values must be reconciled with the approved contract in OD-006/OD-007.

B.1 Untrusted extraction result


```
{
  "request_id": "req-example-001",
  "intents": [
    {
      "name": "CREATE_TASK",
      "confidence": null,
      "slots": {
        "title": {
          "value": "Bài tập OOP",
          "raw_expression": "bài tập OOP",
          "provenance": "USER_MESSAGE"
        },
        "deadline_date": {
          "value": null,
          "raw_expression": null,
          "provenance": null
        },
        "estimated_minutes": {
          "value": 240,
          "raw_expression": "4 tiếng",
          "provenance": "USER_MESSAGE"
        }
      },
      "missing_fields": ["deadline_date"],
      "ambiguities": []
    }
  ]
}
```


B.2 Validated command — identity is server context, not model output


```
{
  "operation": "create_task_candidate",
  "arguments": {
    "title": "Bài tập OOP",
    "estimated_minutes": 240,
    "deadline_date": null
  },
  "validation_status": "NEEDS_CLARIFICATION",
  "trusted_context_ref": "server-managed-context-reference"
}
```

B.3 Proposed plan result with unscheduled work


```
{
 "result_type": "PROPOSAL",
 "plan": {
  "plan_id": "server-generated-id",
  "version": 1,
  "status": "PROPOSED",
  "period_start": "2026-11-09",
  "period_end_exclusive": "2026-11-16",
  "blocks": [],
  "unscheduled_work": [
   {
    "task_id": "server-resolved-task-id",
    "unscheduled_minutes": 120,
    "reason_code": "INSUFFICIENT_AVAILABLE_CAPACITY"
   }
  ]
 },
 "requires_approval": true
}
```


B.4 Error envelope


```
{
 "status": "ERROR",
 "error": {
  "code": "STALE_PLAN_VERSION",
  "message": "The proposal is no longer current. Please review an updated proposal.",
  "details": {
   "proposal_id": "server-generated-id"
  },
  "retryable": false
 },
 "correlation_id": "server-generated-correlation-id"
}
```


Appendix C — Status taxonomy

Domain	Status/value	Meaning	Must not be confused with
Validation	READY	Required values are structurally/semantically/domain valid for the next operation.	User approval or persistence.
Validation	NEEDS_CLARIFICATION	A missing/ambiguous value blocks safe continuation.	Invalid data necessarily.
Validation	UNSUPPORTED	Request is outside currently supported scope.	Backend error.
Validation	INVALID	Supplied value violates schema/business constraints.	Clarification when user can correct it.

Domain	Status/value	Meaning	Must not be confused with
Workflow	AWAITING_CLARIFICATION	PendingAction is waiting for user input.	Entity status.
Workflow	AWAITING_APPROVAL	One identified proposal awaits student decision.	Confirmed plan.
Workflow	COMMITTED	Backend confirms persistence.	LLM prediction of success.
Proposal	PROPOSED	Candidate schedule change for review.	CONFIRMED.
Proposal	REJECTED / EXPIRED / SUPERSEDED	Proposal is no longer eligible for approval.	Deleted historical record unless retention policy says so.
Domain	DRAFT / CANDIDATE	Task data has not become a confirmed task.	Scheduler eligible task.
Domain	CONFIRMED	Student-confirmed task/plan/block according to entity lifecycle.	Merely extracted or proposed data.

Enum spelling and complete state sets must be frozen in OD-006/OD-009 before code generation. This table expresses the distinctions required by the source design, not a final enum contract.

Appendix D — Scheduler invariants and verification formulas

For half-open intervals $A = [s_A, e_A)$ and $B = [s_B, e_B)$, the intervals overlap exactly when:

$$s_A < e_B \text{ AND } s_B < e_A$$

Consequently, $[08:00, 09:00)$ and $[09:00, 10:00)$ are adjacent but do not overlap. A scheduled block must satisfy, at minimum:

$$\text{block.start} < \text{block.end}$$
$$\text{block} \subseteq \text{planning_period}$$
$$\text{block} \subseteq \text{task_eligibility_window}$$
$$\text{block} \subseteq \text{student_availability}$$
$$\text{block} \cap \text{busy_intervals} = \varnothing$$
$$\text{block} \cap \text{other_confirmed_blocks} = \varnothing$$
$$\text{block.end} <= \text{applicable_deadline_boundary}$$
$$\text{sum(new_allocations_for_task)} <= \text{approved_remaining_minutes}$$

The exact endpoint convention for a date-only deadline, block granularity, minimum/maximum block length, breaks, daily caps and task remaining-effort update rule remain TBD. These must be configured and tested before deterministic scheduling is considered release-ready.

Appendix E — Assumption and TBD register

ID	Assumption / TBD	Current treatment
AT-001	User interacts as an authenticated student.	Assumed from target user and authorization requirements; auth provider TBD.
AT-002	The Initial Release is a web application.	Baseline in proposal; exact compatibility matrix TBD.
AT-003	Supported course PDFs are text-extractable.	Baseline; image-only/OCR-heavy files fall back to manual entry.
AT-004	Timezone may default to Asia/Ho_Chi_Minh for the current project context.	Context reference only; product timezone must come from trusted configuration/user profile.
AT-005	Weekly planning period boundaries.	Week start, local clock and inclusive/exclusive end must be approved.
AT-006	Storage schema and database engine.	TBD; logical entities are specified without vendor choice.
AT-007	Backend/API framework and transport.	TBD.
AT-008	LLM/NLU provider/model/version.	TBD pending comparative benchmark and data-policy review.
AT-009	File size/page limits and retention.	TBD; must be configured before public upload.
AT-010	Numeric SLOs and evaluation thresholds.	TBD; no benchmark evidence supplied.

ID	Assumption / TBD	Current treatment
AT-011	PendingAction expiry duration.	TBD; no expiry interval invented.
AT-012	Approval channel and expiry.	TBD; approval must still be bound to user/proposal/version.
AT-013	Recurrence support beyond NONE/WEEKLY.	Deferred/TBD.
AT-014	Separate FixedCommitment entity.	TBD; busy intervals must be represented somehow in scheduler input.
AT-015	Whether update/delete/manual scheduling are MVP.	TBD; treated as deferred by default in this SRS.

Appendix F — Source register and limitations

1. **Team03_Proposal_Updated(1).pdf**, *Midterm Engineering Report — StudyFlow AI*, October 2026, 28 pages. Primary source for problem framing, target users, release scope, feature descriptions, proposed workflows, architecture, conceptual data model, UI mockups, team roles and roadmap. The proposal itself states that mockups and proposed workflows are not evidence of implementation or measured results.
2. **context(2).md**, *StudyFlow AI — Context tiếp nối thiết kế F1*, version 1.1, 04 November 2026. Source for prior conceptual design of intents, parameter schema, tool catalog, orchestrator pipeline, data entities, validation and unresolved decisions. It explicitly says the underlying 0-main-20.pdf had not been directly read at the time that context was prepared; the newly supplied updated proposal is now used directly as the primary project proposal for this SRS.
3. **cong_viec_can_lam(2).md**, *StudyFlow AI — Các công việc cần làm hiện tại*, 04 November 2026. Source for P0/P1/P2 tasks, dependencies, outputs, proposed completion criteria and the observation that no implementation/benchmark evidence was available in the workspace at that date.
4. **StudyFlow_AI_SRS_Master_Prompt(1).pdf**, master prompt supplied by the user. Used as the requested structure and coverage checklist, adapted where the updated proposal provides more authoritative scope detail.

Source limitations

- The preliminary survey values in the proposal (n=17) cannot be independently audited from supplied files because the questionnaire and anonymized response data are not included.
- The related external studies/products listed in the proposal were not independently re-researched for this SRS; this document preserves their role as background rather than asserting fresh validation.
- No source supplied a finalized database engine, programming language/framework, API protocol/path, LLM/NLU provider/model/version, numeric performance target, approved evaluation threshold, file retention policy, authentication provider, or approved release date.
- The SRS is a detailed engineering draft, not a claim that the system has been implemented, benchmarked, tested or approved.

Appendix G — Final engineering readiness checklist

Before using this document as an autonomous coding specification, the team should sign off the following gates:

- ☐ Confirm F1 MVP scope and the P2 deferrals in OD-001..OD-005.
- ☐ Freeze I/O schemas, canonical field names and enum sets (OD-006/OD-007).
- ☐ Approve tool contracts and dispatch allowlist (OD-008).
- ☐ Approve PendingAction and multi-intent state transitions (OD-009/OD-010).
- ☐ Approve timezone, deadline boundary and scheduling policy constants (OD-011/OD-012).
- ☐ Approve partial scheduling semantics (OD-013).
- ☐ Approve approval evidence, transaction, concurrency, idempotency and recovery (OD-014/OD-015).
- ☐ Freeze evaluation data, acceptance thresholds and supported environment (OD-016/OD-020).
- ☐ Decide stack/model/file policy and record architecture decisions (OD-017..OD-019).
- ☐ Assign owners/reviewers/dates outside this SRS (OD-021).
- ☐ Implement and run the P1 test suite; attach actual test/benchmark evidence before declaring MVP complete.

Final status: This SRS is detailed enough to guide architecture, contract design, test design and F1 implementation planning. It is not yet a fully frozen autonomous code-generation baseline because the P0 decisions above remain open. A code-generation agent must treat all TBD, PROPOSED, REQUIRES REVIEW and deferred items as constraints: it may scaffold interfaces, tests and configurable policies, but must not silently choose unresolved product behavior.